Topic 1: Database Fundamentals (Q1–Q20)

Q1. What is data?

Answer: Data is a collection of raw facts, values, or observations that have not necessarily been processed or organized into meaningful information.

Example: 101, Mallika, 85 are individual data values representing a student ID, name, and marks.

Q2. What is a database?

Answer: A database is an organized collection of related data stored electronically so that it can be accessed, managed, updated, and retrieved efficiently.

Example: A college database can store student details, course information, faculty records, and examination marks.

Q3. What is a DBMS?

Answer: DBMS stands for Database Management System. It is software used to create, store, retrieve, update, and manage data in databases.

Examples: MySQL, Oracle Database, PostgreSQL, and Microsoft SQL Server.

Q4. What is the difference between data and information?

Answer:

- Data: Raw facts that may not have a meaningful interpretation by themselves.
- Information: Data that has been processed or organized to provide meaning.

Example:

- Data: 70, 80, 90
- Information: The average marks of a student are 80.

Q5. What is the difference between a database and a DBMS?

Answer:

- Database: The organized collection of stored data.
- DBMS: The software used to manage that data.

Example: A college's student records form the database, while MySQL is the DBMS used to manage those records.

Q6. What is the difference between DBMS and RDBMS?

Answer:

- DBMS: A general term for software that manages databases using different data models.
- RDBMS: A type of DBMS that organizes data primarily into related tables consisting of rows and columns.

An RDBMS supports relationships between tables and commonly uses keys and constraints to maintain data integrity.

Examples: MySQL, PostgreSQL, and Oracle Database are relational database management systems.

Q7. What is a table in a database?

Answer: A table is a database structure that organizes related data into rows and columns.

Example:

Student_ID	Name	Marks
101	Ravi	85
102	Priya	92
103	Arun	78

Here, Students could be the table name.

Q8. What is a field in a database?

Answer: A field generally refers to an individual data attribute represented by a column in a table.

Example: In a Students table, Student_ID, Name, and Marks are fields or columns.

Q9. What is a record in a database?

Answer: A record is a complete set of related field values represented by one row in a table.

Example: 101, Ravi, 85 represents one student record.

Q10. What is the difference between a field and a record?

Answer:

- Field: Represents one attribute, generally a column.
- Record: Represents one complete entry, generally a row.

Example: In a student table, Name is a field, while 101, Ravi, 85 is a record.

Q11. What is the difference between a row and a column?

Answer:

- Row: Represents one record containing values for multiple attributes.
- Column: Represents an attribute shared across records.

Example: In a student table, one row contains Ravi's details, while the Name column contains the names of all students.

Q12. What is a database schema?

Answer: A database schema describes the logical structure of a database, including tables, columns, data types, relationships, and constraints.

Example: A student management schema may define Students, Courses, and Enrollments tables with primary and foreign keys.

Q13. What is metadata in a database?

Answer: Metadata is data that describes the structure and properties of other data.

Examples include:

- Table names.
- Column names and data types.
- Primary and foreign key definitions.
- Index information.
- Constraints and other database object definitions.

In MySQL, metadata can be inspected through INFORMATION_SCHEMA and commands such as DESCRIBE.

Q14. What is a relational database?

Answer: A relational database organizes data into tables and establishes relationships between tables using keys and constraints.

Example: A Students table and a Courses table can be connected through an Enrollments table that records which students enroll in which courses.

This structure reduces unnecessary duplication and supports queries across related data.

Q15. What are the advantages of using a DBMS?

Answer:

The main advantages are:

1. Reduced unnecessary data redundancy.

2. Improved data consistency and integrity.
3. Controlled access and security.
4. Data sharing among multiple users.
5. Backup and recovery support.
6. Transaction management.
7. Efficient data retrieval using queries and indexes.
8. Better data organization and maintenance.

Q16. What is data redundancy?

Answer: Data redundancy means storing the same data unnecessarily in multiple places.

Example: If a student's address is duplicated across several tables, updating only one copy can result in inconsistent information.

Database normalization and appropriate table relationships help reduce unnecessary redundancy.

Q17. What is data integrity?

Answer: Data integrity means maintaining the accuracy, validity, and consistency of data throughout its lifecycle.

Example: A foreign key can prevent an enrollment record from referring to a student who does not exist in the referenced student table.

Primary keys, unique constraints, check constraints, foreign keys, and transactions help maintain integrity.

Q18. What is SQL?

Answer: SQL stands for Structured Query Language. It is the language used to define, retrieve, manipulate, and manage data in relational databases.

Examples of SQL commands:

- CREATE — creates database objects.
- INSERT — adds rows.
- SELECT — retrieves data.
- UPDATE — modifies data.
- DELETE — removes rows.

Q19. What is MySQL?

Answer: MySQL is a relational database management system that uses SQL to manage relational data.

It is widely used in web applications, enterprise applications, and backend systems.

Example: A Spring Boot application can use MySQL to store users, products, orders, and payment records.

Q20. What is the difference between SQL and MySQL?

Answer:

SQL	MySQL
A query language	A relational database management system
Defines statements used to work with data	Executes SQL statements to manage data
Has syntax and standards implemented by different databases	Supports SQL with MySQL-specific features and syntax
Is not a database product itself	Is a database product

Example: SELECT * FROM Students; is an SQL statement that can be executed in MySQL.

Topic 2: Keys and Constraints (Q21–Q32)

Q21. What is a key in a database?

Answer: A key is an attribute or combination of attributes used to identify records uniquely or establish relationships between tables.

Examples: Primary key, foreign key, candidate key, composite key, and alternate key.

Q22. What is a primary key?

Answer: A primary key is a column or combination of columns that uniquely identifies each row in a table.

Its values must be unique and cannot be NULL.

Example:

```
CREATE TABLE Students (
   student_id INT PRIMARY KEY,
   name VARCHAR(50)
);
```

Here, student_id uniquely identifies each student.

Q23. What is a foreign key?

Answer: A foreign key is a column or combination of columns that references a primary key or another eligible unique key in a referenced table.

It helps enforce referential integrity between related tables.

Example:

```
CREATE TABLE Enrollments (
   enrollment_id INT PRIMARY KEY,
   student_id INT,
   FOREIGN KEY (student_id)
      REFERENCES Students(student_id)
);
```

The foreign key ensures that a non-null student_id in Enrollments references an existing student, subject to the defined referential actions.

Q24. What is a candidate key?

Answer: A candidate key is a minimal set of columns that uniquely identifies each row in a table.

A table can have multiple candidate keys, and one of them is selected as the primary key.

Example: In an employee table, both employee_id and official_email could be candidate keys if each is unique, non-null, and independently sufficient to identify an employee.

Q25. What is a composite key?

Answer: A composite key is a key formed using two or more columns together to identify a row uniquely.

Example: In an enrollment table, the combination of student_id and course_id can form a composite primary key if a student can enroll in a particular course only once.

PRIMARY KEY (student_id, course_id)

Neither column must necessarily be unique by itself; the combination must be unique.

Q26. What is the difference between a primary key and a unique key?

Answer:

Primary key	**Unique key**

Uniquely identifies each row	Prevents duplicate key combinations
Cannot contain NULL	MySQL permits multiple NULL values in a unique index, subject to index design
Only one primary key constraint per table	Multiple unique constraints can exist
Commonly used as the main row identifier	Commonly enforces uniqueness for other attributes

Example: An employee table may use employee_id as its primary key and official_email as a unique key.

Q27. What is the difference between a primary key and a foreign key?

Answer:

- Primary key: Uniquely identifies a row in its own table.
- Foreign key: References a key in another table, or a suitable key in the same table, to maintain a valid relationship.

Example: Students.student_id may be a primary key, while Enrollments.student_id is a foreign key referencing it.

A table can have one primary key constraint and multiple foreign key constraints.

Q28. What are constraints in MySQL?

Answer: Constraints are rules applied to columns or tables to help maintain valid and consistent data.

Common MySQL constraints include:

- PRIMARY KEY
- FOREIGN KEY
- UNIQUE
- NOT NULL
- CHECK
- DEFAULT (a column default definition)

Example: A CHECK (marks BETWEEN 0 AND 100) constraint can restrict marks to the valid range.

Q29. What is the purpose of the NOT NULL constraint?

Answer: The NOT NULL constraint prevents a column from storing NULL values.

Example:

name VARCHAR(50) NOT NULL

Every inserted row must provide a non-null name, or the column must receive a valid non-null default.

Important: NOT NULL does not prevent an empty string such as ''.

Q30. What is the purpose of the UNIQUE constraint?

Answer: The UNIQUE constraint prevents duplicate values for a column or duplicate combinations of values across multiple columns.

Example:

email VARCHAR(100) UNIQUE

This prevents two rows from having the same non-null email value. In MySQL, a unique index can generally contain multiple NULL values.

Q31. What is the purpose of the DEFAULT constraint?

Answer: A default definition specifies the value MySQL uses when an INSERT statement omits the column or explicitly uses the DEFAULT keyword.

Example:

status VARCHAR(20) DEFAULT 'Active'

If an insert omits status, MySQL uses 'Active'.

Important: An explicit NULL is not automatically replaced by the default. Whether it is accepted depends on the column definition and SQL mode.

Q32. What is the purpose of the CHECK constraint?

Answer: A CHECK constraint restricts values according to a specified condition.

Example:

marks INT CHECK (marks BETWEEN 0 AND 100)

This constraint rejects values outside the allowed range when the condition evaluates to false.

In MySQL 8.0.16 and later, CHECK constraints are enforced. A check condition that evaluates to UNKNOWN because of NULL does not itself reject the row, so use NOT NULL when nulls must also be prohibited.

Topic 3: SQL Commands and Data Types (Q33–Q44)

Q33. What are the different types of SQL commands?

Answer:

SQL commands are commonly grouped into five categories:

1. DDL (Data Definition Language): Defines database structures.
2. DML (Data Manipulation Language): Inserts, modifies, or deletes data.
3. DQL (Data Query Language): Retrieves data.
4. DCL (Data Control Language): Controls access and privileges.
5. TCL (Transaction Control Language): Manages transactions.

Examples include CREATE, INSERT, SELECT, GRANT, and COMMIT, respectively.

Q34. What is the difference between DDL, DML, DQL, DCL and TCL?

Answer:

Category	Purpose	Common commands
DDL	Defines or changes database structures	CREATE, ALTER, DROP, TRUNCATE
DML	Manipulates table data	INSERT, UPDATE, DELETE
DQL	Retrieves data	SELECT
DCL	Manages privileges	GRANT, REVOKE
TCL	Manages transactions	START TRANSACTION, COMMIT, ROLLBACK, SAVEPOINT

Example: Creating a table is DDL, inserting a student is DML, retrieving students is DQL, granting access is DCL, and committing a transaction is TCL.

Interview point: Some classifications place SELECT under DML, while many training materials treat DQL as a separate category. Follow the classification expected in the interview.

Q35. What is the difference between DELETE, DROP and TRUNCATE?

Answer:

Feature	DELETE	TRUNCATE	DROP

Purpose	Removes selected or all rows	Removes all rows	Removes the table itself
WHERE clause	Supported	Not supported	Not supported
Table structure	Preserved	Preserved	Removed
Command category	DML	Commonly classified as DDL in MySQL	DDL
Auto-increment counter	Generally not reset by deleting rows	Reset to its initial value	Table and its counter are removed
Rollback	Depends on transaction and storage engine	Causes an implicit commit in MySQL	Causes an implicit commit in MySQL

Examples:

DELETE FROM Students WHERE student_id = 101;

TRUNCATE TABLE Students;

DROP TABLE Students;

The first removes a matching student, the second removes all rows, and the third removes the table.

Q36. What is the difference between CHAR and VARCHAR?

Answer:

Both data types store character strings, but they differ in how their lengths are handled.

- CHAR(n): Fixed-length character type. Values are padded to the declared length as needed.
- VARCHAR(n): Variable-length character type. It stores values up to the declared maximum length, with additional length-storage overhead.

Example:

gender CHAR(1),

name VARCHAR(50)

CHAR(1) is suitable for a single-character code, while VARCHAR(50) is suitable for names of varying lengths.

Interview point: The maximum length and storage behavior depend on character set, byte limits, and MySQL's type rules. CHAR is not automatically faster for every use case.

Q37. What are the commonly used data types in MySQL?

Answer:

MySQL supports several categories of data types.

Category	Common types	Purpose
Numeric	INT, BIGINT, DECIMAL, FLOAT, DOUBLE	Whole numbers and decimal values
String	CHAR, VARCHAR, TEXT	Character data
Date and time	DATE, TIME, DATETIME, TIMESTAMP, YEAR	Dates and times
Boolean-like	BOOLEAN, BOOL	Aliases for TINYINT(1)
Binary	BINARY, VARBINARY, BLOB	Binary data
JSON	JSON	Structured JSON documents

Example: An employee table may use INT for employee ID, VARCHAR(100) for name, DECIMAL(10,2) for salary, and DATE for joining date.

Interview point: DECIMAL is generally preferred for monetary amounts requiring exact decimal arithmetic.

Q38. What is the purpose of the CREATE command?

Answer:

The CREATE command is a DDL statement used to create database objects, including databases, tables, views, and indexes.

Example:

```
CREATE DATABASE college;


CREATE TABLE Students (
   student_id INT PRIMARY KEY,
   name VARCHAR(50),
   marks INT
```

);

The first statement creates a database, and the second creates a table within the currently selected database.

Q39. What is the difference between ALTER and UPDATE?

Answer:

- ALTER: Changes the structure or definition of a database object.
- UPDATE: Changes existing row values in a table.

Example of ALTER:

```
ALTER TABLE Students
ADD COLUMN email VARCHAR(100);
```

Example of UPDATE:

```
UPDATE Students
SET marks = 90
WHERE student_id = 101;
```

The first adds a column, while the second changes a student's marks.

Q40. What is the purpose of the INSERT command?

Answer:

The INSERT statement adds new rows to a table.

Example:

```
INSERT INTO Students (student_id, name, marks)
VALUES (101, 'Ravi', 85);
```

Multiple rows can also be inserted in one statement:

```
INSERT INTO Students (student_id, name, marks)
VALUES
  (102, 'Priya', 92),
  (103, 'Arun', 78);
```

Q41. What is the purpose of the SELECT command?

Answer:

The SELECT statement retrieves data from one or more tables or other queryable sources.

Example:

SELECT student_id, name, marks

FROM Students;

To retrieve all columns, you can use:

SELECT * FROM Students;

In production applications, specifying the required columns is often preferable because it makes queries clearer and reduces unnecessary data retrieval.

Q42. What is the purpose of the WHERE clause?

Answer:

The WHERE clause filters rows according to a specified condition.

Example:

SELECT *

FROM Students

WHERE marks > 80;

This returns only students whose marks are greater than 80.

The WHERE clause can use comparison operators, logical operators, LIKE, IN, BETWEEN, and other valid conditions.

Q43. What is the purpose of the ORDER BY clause?

Answer:

The ORDER BY clause sorts the result set according to one or more columns.

- ASC sorts in ascending order and is the default.
- DESC sorts in descending order.

Example:

SELECT name, marks

FROM Students

ORDER BY marks DESC;

This displays students from highest marks to lowest marks.

Multiple columns can be specified to define secondary sorting criteria.

Q44. What is the purpose of the DISTINCT keyword?

Answer:

The DISTINCT keyword removes duplicate rows from the query result based on the selected expressions.

Example:

SELECT DISTINCT city

FROM Students;

This returns each distinct city only once.

If multiple columns are selected, DISTINCT applies to the combination of selected column values, not independently to each column.

Topic 4: SQL Clauses and Functions (Q45–Q60)

Q45. What is the purpose of the GROUP BY clause?

Answer:

The GROUP BY clause groups rows that have the same values in specified columns. It is commonly used with aggregate functions such as COUNT(), SUM(), and AVG().

Example:

SELECT city, COUNT(*) AS total_students

FROM Students

GROUP BY city;

This query returns the number of students in each city.

Q46. What is the difference between WHERE and HAVING?

Answer:

- WHERE: Filters individual rows before grouping and aggregation.
- HAVING: Filters groups after grouping and aggregation.

Example:

SELECT city, AVG(marks) AS average_marks

FROM Students

WHERE marks >= 40

GROUP BY city

HAVING AVG(marks) > 75;

Here, WHERE excludes students with marks below 40 before grouping. HAVING returns only cities whose remaining students have an average above 75.

Q47. What is the difference between GROUP BY and ORDER BY?

Answer:

- GROUP BY: Groups rows that share the same specified values, commonly for aggregate calculations.
- ORDER BY: Sorts the final query results.

Example:

SELECT city, COUNT(*) AS total_students

FROM Students

GROUP BY city

ORDER BY total_students DESC;

This groups students by city and sorts the cities from the highest student count to the lowest.

Q48. What is a function in SQL?

Answer:

A SQL function is a built-in or user-defined operation that accepts input values, performs a calculation or transformation, and returns a result.

Common categories include:

- Aggregate functions: COUNT(), SUM(), AVG().
- String functions: UPPER(), LOWER(), CONCAT().
- Numeric functions: ROUND(), ABS().
- Date and time functions: CURDATE(), YEAR(), DATEDIFF().

Example:

SELECT UPPER('mysql') AS result;

Output: MYSQL

Q49. What are aggregate functions in MySQL?

Answer:

Aggregate functions calculate a result from multiple rows and return a single result for the selected group.

Common aggregate functions are:

- COUNT() — counts rows or non-null values.
- SUM() — calculates a total.
- AVG() — calculates an average.
- MIN() — finds the minimum value.
- MAX() — finds the maximum value.

Example:

```
SELECT
  COUNT(*) AS total_students,
  SUM(marks) AS total_marks,
  AVG(marks) AS average_marks,
  MIN(marks) AS lowest_marks,
  MAX(marks) AS highest_marks
FROM Students;
```

Without GROUP BY, this calculates the aggregate values over the qualifying rows as a whole.

Q50. What is the purpose of the COUNT() function?

Answer:

The COUNT() function counts rows or non-null values, depending on the expression supplied.

Example:

```
SELECT COUNT(*) AS total_students
FROM Students;
```

This counts all rows returned by the query, including rows containing NULL in some columns.

Another example:

```
SELECT COUNT(email) AS students_with_email
FROM Students;
```

This counts only rows where email is not NULL.

Q51. What is the difference between COUNT(*) and COUNT(column_name)?

Answer:

COUNT(*)	COUNT(column_name)
Counts all rows in the qualifying result	Counts rows where the specified column is non-null
Includes rows with null column values	Excludes rows where that column is NULL
Useful for total row counts	Useful for counting available values

Example: If a table contains five students and two have a NULL email address, COUNT(*) returns 5, while COUNT(email) returns 3.

Q52. What is the purpose of the SUM() function?

Answer:

The SUM() function calculates the total of numeric values in a column or expression, ignoring NULL values.

Example:

```
SELECT SUM(marks) AS total_marks
FROM Students;
```

This returns the total marks across the qualifying student rows.

If no non-null values are available, SUM() returns NULL.

Q53. What is the purpose of the AVG() function?

Answer:

The AVG() function calculates the arithmetic mean of non-null numeric values.

It adds the non-null values and divides the total by the number of non-null values.

Example:

```
SELECT AVG(marks) AS average_marks
FROM Students;
```

If the marks are 70, 80, and 90, the average is 80.

AVG() ignores NULL values rather than treating them as zero.

Q54. What is the difference between MIN() and MAX()?

Answer:

- MIN() returns the smallest value in the qualifying set.
- MAX() returns the largest value in the qualifying set.

They can be applied to numeric, date, and string expressions, with comparisons following the relevant data type and collation rules.

Example:

```
SELECT
  MIN(marks) AS lowest_marks,
  MAX(marks) AS highest_marks
FROM Students;
```

This returns the lowest and highest non-null marks.

If there are no non-null values, both functions return NULL.

Q55. What is the difference between WHERE and DISTINCT?

Answer:

WHERE filters rows based on a condition, while DISTINCT removes duplicate rows from the selected result.

Example using WHERE:

```
SELECT name, city
FROM Students
WHERE city = 'Hyderabad';
```

This returns students whose city is Hyderabad.

Example using DISTINCT:

```
SELECT DISTINCT city
FROM Students;
```

This returns each distinct city represented in the result.

Interview point: WHERE controls which rows qualify; DISTINCT controls whether duplicate selected rows appear in the output.

Q56. How does MySQL handle NULL values?

Answer:

NULL represents a missing, unknown, or unavailable value. It is not the same as zero or an empty string.

Important rules include:

1. Use IS NULL to check for null values.
2. Use IS NOT NULL to check for non-null values.
3. Comparisons such as column = NULL do not evaluate to true.
4. Most aggregate functions ignore null values.
5. COUNT(*) counts rows regardless of nulls in individual columns.

Example:

SELECT *

FROM Students

WHERE email IS NULL;

This retrieves students whose email value is missing.

Q57. What is the difference between NULL, zero and an empty string?

Answer:

These values have different meanings.

- NULL: Represents missing, unknown, or unavailable data.
- Zero (0): A numeric value representing zero.
- Empty string (''): A string containing zero characters.

Example:

Value	Meaning
NULL	Marks have not been recorded
0	The recorded marks are zero
''	A text field contains an empty string

Important: A query using column = 0 checks for zero, and a query using column = '' checks for an empty string. Use IS NULL to find null values.

Q58. What is the purpose of the LIMIT clause?

Answer:

The LIMIT clause restricts the number of rows returned by a query. It is commonly used for retrieving a small result set or implementing pagination.

Example:

```
SELECT *
FROM Students
LIMIT 5;
```

This returns at most five rows.

To skip the first five rows and return the next five:

```
SELECT *
FROM Students
ORDER BY student_id
LIMIT 5 OFFSET 5;
```

This returns up to five rows after skipping the first five, assuming the result contains enough rows.

Interview point: Use ORDER BY with LIMIT when a predictable ordering is required.

Intermediate Level

Topic 1: Joins and Relationships

Q1. What is a join in MySQL, and why is it required?

Answer: A join combines related rows from two or more tables using a specified condition. Joins help retrieve related information without storing all data in a single table.

For example, an employees table may store department_id, while a departments table stores department names.

```
SELECT e.emp_name, d.department_name
FROM employees e
INNER JOIN departments d
```

ON e.department_id = d.department_id;

This returns employees along with their matching department names.

Q2. What is the difference between INNER JOIN, LEFT JOIN, and RIGHT JOIN?

Answer:

Join	Result
INNER JOIN	Returns rows with matching records in both tables
LEFT JOIN	Returns all left-table rows and matching right-table rows; unmatched right-side columns are NULL
RIGHT JOIN	Returns all right-table rows and matching left-table rows; unmatched left-side columns are NULL

Example:

```
SELECT e.emp_name, d.department_name
FROM employees e
LEFT JOIN departments d
  ON e.department_id = d.department_id;
```

This returns every employee, including employees without a matching department.

Q3. Does MySQL support FULL OUTER JOIN directly? How can you achieve the same result?

Answer: MySQL does not directly support the FULL OUTER JOIN syntax. We can simulate its behavior by combining a LEFT JOIN with a RIGHT JOIN using UNION.

```
SELECT e.emp_name, d.department_name
FROM employees e
LEFT JOIN departments d
  ON e.department_id = d.department_id

UNION

SELECT e.emp_name, d.department_name
```

```
FROM employees e
RIGHT JOIN departments d
  ON e.department_id = d.department_id;
```

UNION removes duplicate result rows. If using UNION ALL, additional filtering is required to avoid returning matching rows twice.

Q4. What is a CROSS JOIN, and when is it useful?

Answer: A CROSS JOIN produces the Cartesian product of two tables. Every row from the first table is combined with every row from the second table.

If one table contains 3 rows and another contains 4 rows, the result contains 12 rows.

```
SELECT s.student_name, c.course_name
FROM students s
CROSS JOIN courses c;
```

This is useful for generating all possible student-course combinations, constructing test data, or building combinations of sizes and colors for products.

Q5. What is a self join, and what are its real-world applications?

Answer: A self join joins a table with itself using different aliases.

It is useful for hierarchical or comparative relationships, such as finding employees and their managers.

```
SELECT e.emp_name AS employee,
    m.emp_name AS manager
FROM employees e
LEFT JOIN employees m
  ON e.manager_id = m.emp_id;
```

Here, both aliases refer to the same employees table. The LEFT JOIN also includes employees who have no manager.

Q6. What is the difference between an equi join and a non-equi join?

Answer:

- Equi join: Uses equality (=) in the join condition.
- Non-equi join: Uses other comparison operators, such as <, >, BETWEEN, or <=.

Equi join example:

```
SELECT e.emp_name, d.department_name
FROM employees e
JOIN departments d
   ON e.department_id = d.department_id;
```

Non-equi join example:

```
SELECT e.emp_name, g.grade
FROM employees e
JOIN salary_grades g
   ON e.salary BETWEEN g.min_salary AND g.max_salary;
```

The second query matches employees to salary grades based on a range.

Q7. What is the difference between a join and a subquery?

Answer: A join combines related rows from multiple tables into a result set, whereas a subquery is a query nested inside another SQL statement.

A join is often useful when columns from multiple tables are required. A subquery is useful for filtering or calculating values based on another query.

Join example:

```
SELECT e.emp_name, d.department_name
FROM employees e
JOIN departments d
   ON e.department_id = d.department_id;
```

Subquery example:

```
SELECT emp_name
FROM employees
WHERE salary > (
   SELECT AVG(salary)
   FROM employees
);
```

Neither approach is universally faster. MySQL's optimizer may transform subqueries into joins or other execution strategies.

Q8. What is the difference between filtering conditions in ON and WHERE when using an outer join?

Answer: A condition in ON determines which rows match during the join. A condition in WHERE filters the result after the join.

Example using ON:

```
SELECT e.emp_name, d.department_name

FROM employees e

LEFT JOIN departments d

   ON e.department_id = d.department_id

  AND d.department_name = 'IT';
```

This returns all employees, but department details are populated only when the matching department is IT.

If the department condition is moved to WHERE, unmatched rows are removed because their department name is NULL. The result behaves like an inner join for that condition.

Q9. How do you find employees who do not belong to any department?

Answer: Use a LEFT JOIN and filter for unmatched department records.

```
SELECT e.*

FROM employees e

LEFT JOIN departments d

   ON e.department_id = d.department_id

WHERE d.department_id IS NULL;
```

This finds employees without a matching department. If the business rule specifically means employees whose department_id is null, use WHERE e.department_id IS NULL instead.

Q10. How do you find records that exist in one table but not in another?

Answer: Use NOT EXISTS to find records from one table for which no matching record exists in the other.

Example: Find students who have not enrolled in any course.

```
SELECT s.*
```

```
FROM students s
WHERE NOT EXISTS (
  SELECT 1
  FROM enrollments e
  WHERE e.student_id = s.student_id
);
```

This is an anti-join pattern. It is generally safer than NOT IN when the subquery may return NULL.

Q11. How do you join three or more tables in a single query?

Answer: Use multiple join clauses, specifying the relationship between each pair of tables.

```
SELECT s.student_name,
    c.course_name,
    e.enrollment_date
FROM students s
JOIN enrollments e
  ON s.student_id = e.student_id
JOIN courses c
  ON e.course_id = c.course_id;
```

This returns student names, course names, and enrollment dates by joining three related tables.

Q12. What is a many-to-many relationship, and how is it implemented using a junction table?

Answer: A many-to-many relationship occurs when multiple records in one table can be associated with multiple records in another table.

For example, a student can enroll in multiple courses, and each course can contain multiple students.

A junction table, such as enrollments, stores the relationship using foreign keys.

```
CREATE TABLE enrollments (
  student_id INT,
```

```
    course_id INT,
    enrollment_date DATE,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id)
        REFERENCES students(student_id),
    FOREIGN KEY (course_id)
        REFERENCES courses(course_id)
);
```

The composite primary key prevents the same student-course combination from being entered more than once.

Topic 2: Subqueries and Common Table Expressions

Q13. What is a subquery, and what are its different types?

Answer: A subquery is a query nested inside another SQL statement.

Common classifications include:

- Scalar subquery: Returns a single value.
- Row subquery: Returns one row containing multiple columns.
- Table subquery: Returns multiple rows and possibly multiple columns.
- Correlated subquery: References columns from the outer query.
- Non-correlated subquery: Executes logically independently of the outer query.

Subqueries can appear in SELECT, FROM, WHERE, and HAVING clauses, subject to SQL syntax rules.

Q14. What is the difference between correlated and non-correlated subqueries?

Answer:

A non-correlated subquery does not depend on the outer query's current row. A correlated subquery references a value from the outer query.

Non-correlated example:

```
SELECT emp_name
FROM employees
WHERE salary > (
```

```
    SELECT AVG(salary)
    FROM employees
);
```

Correlated example:

```
SELECT e.emp_name, e.salary
FROM employees e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.department_id = e.department_id
);
```

The correlated example compares each employee's salary with the average salary of that employee's department.

Q15. What is the difference between IN and EXISTS?

Answer:

- IN checks whether a value matches any value in a specified list or subquery result.
- EXISTS checks whether a subquery returns at least one row.

Example using IN:

```
SELECT emp_name
FROM employees
WHERE department_id IN (
    SELECT department_id
    FROM departments
    WHERE department_name = 'IT'
);
```

Example using EXISTS:

```
SELECT d.department_name
FROM departments d
```

```
WHERE EXISTS (
  SELECT 1
  FROM employees e
  WHERE e.department_id = d.department_id
);
```

Both can be efficient, depending on the query and indexes. NOT EXISTS is often preferable to NOT IN when null values are possible in the subquery.

Q16. What is the difference between ANY and ALL operators?

Answer:

- ANY returns true if the comparison is true for at least one value returned by the subquery.
- ALL returns true if the comparison is true for every value returned by the subquery.

Example:

```
SELECT emp_name, salary
FROM employees
WHERE salary > ANY (
  SELECT salary
  FROM employees
  WHERE department_id = 10
);
```

This finds employees whose salaries are greater than at least one salary in department 10.

```
SELECT emp_name, salary
FROM employees
WHERE salary > ALL (
  SELECT salary
  FROM employees
  WHERE department_id = 10
);
```

This finds employees whose salaries are greater than every salary in department 10.

Be aware that nulls can make comparisons evaluate to UNKNOWN, and ALL over an empty subquery is true while ANY over an empty subquery is false.

Q17. What is a scalar subquery?

Answer: A scalar subquery returns exactly one column and at most one row, producing a single value or NULL if no row is returned.

Example:

```
SELECT emp_name, salary
FROM employees
WHERE salary > (
   SELECT AVG(salary)
   FROM employees
);
```

The subquery calculates one average salary value. A scalar subquery that returns more than one row causes an error in a scalar context.

Q18. What is a Common Table Expression (CTE)?

Answer: A Common Table Expression is a named temporary result set defined using the WITH clause. It can be referenced within the SQL statement that defines it.

Example:

```
WITH high_salary AS (
   SELECT emp_id, emp_name, salary
   FROM employees
   WHERE salary > 70000
)
SELECT *
FROM high_salary;
```

CTEs improve readability by separating complex query logic into named steps. They are not automatically stored as permanent tables.

Q19. What is the difference between a CTE, a derived table, and a subquery?

Answer:

Feature	CTE	Derived table	Subquery
Definition	Named result set using WITH	Query in the FROM clause	General nested query
Reuse	Can be referenced multiple times in the statement	Usually referenced through its alias	Depends on placement and query structure
Readability	Useful for multi-step queries	Useful for localized transformations	Useful for filtering and scalar calculations
Recursion	Supported with recursive CTE syntax	Not inherently recursive	Not inherently recursive

A derived table is itself a type of subquery.

Q20. What is a recursive CTE, and where can it be used?

Answer: A recursive CTE repeatedly references its own result to build a sequence or traverse hierarchical data. It is defined using WITH RECURSIVE.

Example: Generate numbers from 1 to 5.

```
WITH RECURSIVE numbers AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n + 1
  FROM numbers
  WHERE n < 5
)
SELECT n
FROM numbers;
```

Recursive CTEs are useful for employee-manager hierarchies, category trees, organizational structures, and date or number sequences.

Q21. Can a subquery return multiple rows when used with the equality operator?

Answer: Not in a scalar comparison such as column = (subquery). If the subquery returns multiple rows, MySQL raises an error.

Incorrect example:

```
SELECT *
FROM employees
WHERE department_id = (
   SELECT department_id
   FROM departments
);
```

If the subquery returns multiple rows, this fails.

Use IN when matching any returned value:

```
SELECT *
FROM employees
WHERE department_id IN (
   SELECT department_id
   FROM departments
);
```

Q22. How do you find employees earning more than the average salary using a subquery?

Answer: Calculate the average salary in a subquery and compare each employee's salary against it.

```
SELECT emp_id, emp_name, salary
FROM employees
WHERE salary > (
   SELECT AVG(salary)
   FROM employees
);
```

The query returns employees earning above the overall average salary. Employees with null salaries are not returned by the comparison.

Q23. How would you rewrite a correlated subquery using a join or CTE?

Answer: A correlated subquery can often be rewritten using a grouped CTE and a join.

Original approach:

```
SELECT e.emp_name, e.salary
```

```
FROM employees e
WHERE e.salary > (
   SELECT AVG(e2.salary)
   FROM employees e2
   WHERE e2.department_id = e.department_id
);
```

Rewritten using a CTE:

```
WITH department_avg AS (
   SELECT department_id,
        AVG(salary) AS avg_salary
   FROM employees
   GROUP BY department_id
)
SELECT e.emp_name, e.salary
FROM employees e
JOIN department_avg d
   ON e.department_id = d.department_id
WHERE e.salary > d.avg_salary;
```

This makes the departmental aggregation explicit. The rewrite is not necessarily faster in every case, so compare execution plans using EXPLAIN when performance matters.

Topic 3: Practical SQL Query-Writing Questions

Q24. How do you find the highest and lowest salaries in an employee table?

Answer: Use the aggregate functions MAX() and MIN().

```
SELECT MAX(salary) AS highest_salary,
     MIN(salary) AS lowest_salary
FROM employees;
```

MAX() returns the highest non-null salary, while MIN() returns the lowest non-null salary.

Q25. How do you find the second-highest distinct salary without using LIMIT?

Answer: Use a subquery to find the maximum salary that is less than the highest salary.

```
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (
  SELECT MAX(salary)
  FROM employees
);
```

This finds the second-highest distinct salary, even if multiple employees receive the highest salary.

Q26. How do you find the third-highest distinct salary?

Answer: Use nested subqueries to exclude the highest and second-highest distinct salaries.

```
SELECT MAX(salary) AS third_highest_salary
FROM employees
WHERE salary < (
  SELECT MAX(salary)
  FROM employees
  WHERE salary < (
    SELECT MAX(salary)
    FROM employees
  )
);
```

If fewer than three distinct non-null salaries exist, the result is NULL.

Q27. How do you find the Nth-highest salary in a table?

Answer: Use DENSE_RANK() to rank distinct salary values in descending order.

```
WITH ranked_salaries AS (
  SELECT salary,
      DENSE_RANK() OVER (
        ORDER BY salary DESC
```

```
        ) AS salary_rank
    FROM employees
    WHERE salary IS NOT NULL
)
SELECT DISTINCT salary
FROM ranked_salaries
WHERE salary_rank = 4;
```

This example returns the fourth-highest distinct salary. Replace 4 with the required rank.

DENSE_RANK() gives equal salaries the same rank without gaps.

Q28. How do you find duplicate records based on an email column?

Answer: Use GROUP BY and HAVING COUNT(*) > 1.

```
SELECT email, COUNT(*) AS occurrences
FROM employees
WHERE email IS NOT NULL
GROUP BY email
HAVING COUNT(*) > 1;
```

This returns email values appearing more than once, excluding null emails.

Q29. How do you delete duplicate records while retaining one copy?

Answer: If each employee has a unique emp_id, retain the row with the smallest ID for each email and delete the other rows.

First, identify the duplicates:

```
SELECT email, COUNT(*) AS occurrences
FROM employees
WHERE email IS NOT NULL
GROUP BY email
HAVING COUNT(*) > 1;
```

Then, after backing up the data and verifying the affected rows, delete duplicates:

```
DELETE e1
```

```
FROM employees e1
JOIN employees e2
   ON e1.email = e2.email
  AND e1.emp_id > e2.emp_id;
```

This retains the row with the smallest emp_id for each duplicated non-null email.

Important: Decide how nulls and case sensitivity should be handled, and check foreign key dependencies before deleting production records.

Q30. How do you find employees earning more than their department's average salary?

Answer: Calculate the average salary for each department and compare each employee's salary with that average.

```
SELECT e.emp_id, e.emp_name, e.salary,
     e.department_id
FROM employees e
JOIN (
   SELECT department_id,
        AVG(salary) AS avg_salary
   FROM employees
   GROUP BY department_id
) d
   ON e.department_id = d.department_id
WHERE e.salary > d.avg_salary;
```

This returns employees whose salaries exceed their respective department's average salary.

Q31. How do you find departments having more than five employees?

Answer: Group employees by department and filter the groups using HAVING.

```
SELECT department_id,
     COUNT(*) AS total_employees
FROM employees
WHERE department_id IS NOT NULL
GROUP BY department_id
```

```
HAVING COUNT(*) > 5;
```

To display department names, join the result with the departments table.

Q32. How do you find the top three highest-paid employees in each department?

Answer: Use ROW_NUMBER() or a ranking function with PARTITION BY.

```
WITH ranked_employees AS (
   SELECT emp_id, emp_name,
          department_id, salary,
          ROW_NUMBER() OVER (
             PARTITION BY department_id
             ORDER BY salary DESC, emp_id
          ) AS rn
   FROM employees
   WHERE salary IS NOT NULL
)
SELECT *
FROM ranked_employees
WHERE rn <= 3;
```

This returns at most three employees per department. ROW_NUMBER() assigns unique positions, so ties may be separated.

If you want to include all employees tied within the top three distinct salary positions, use DENSE_RANK() instead.

Q33. How do you find customers who have never placed an order?

Answer: Use NOT EXISTS to find customers without matching order records.

```
SELECT c.*
FROM customers c
WHERE NOT EXISTS (
   SELECT 1
   FROM orders o
   WHERE o.customer_id = c.customer_id
```

```
);
```

This returns every customer for whom no order exists.

Q34. How do you find employees who earn more than their managers?

Answer: Use a self join to compare each employee's salary with the salary of their manager.

```
SELECT e.emp_name AS employee,
    e.salary AS employee_salary,
    m.emp_name AS manager,
    m.salary AS manager_salary
FROM employees e
JOIN employees m
  ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

The query assumes that manager_id references another employee's emp_id.

Q35. How do you find the latest order for every customer?

Answer: Use ROW_NUMBER() to rank each customer's orders by date.

```
WITH ranked_orders AS (
  SELECT o.*,
      ROW_NUMBER() OVER (
        PARTITION BY customer_id
        ORDER BY order_date DESC, order_id DESC
      ) AS rn
  FROM orders o
)
SELECT *
FROM ranked_orders
WHERE rn = 1;
```

This returns one latest order per customer who has at least one order. The order_id tie-breaker makes the selection deterministic when dates are equal.

Q36. How do you find the highest-paid employee in each department, including ties?

Answer: Use DENSE_RANK() partitioned by department.

```
WITH ranked_employees AS (
   SELECT emp_id, emp_name,
         department_id, salary,
         DENSE_RANK() OVER (
            PARTITION BY department_id
            ORDER BY salary DESC
         ) AS salary_rank
   FROM employees
   WHERE salary IS NOT NULL
)
SELECT *
FROM ranked_employees
WHERE salary_rank = 1;
```

All employees tied for the highest salary within their department are included.

Q37. How do you calculate a running total of employee salaries?

Answer: Use the window function SUM() with an OVER clause.

```
SELECT emp_id, emp_name, salary,
      SUM(salary) OVER (
         ORDER BY emp_id
         ROWS BETWEEN UNBOUNDED PRECEDING
               AND CURRENT ROW
      ) AS running_total
FROM employees;
```

The running total accumulates salaries in emp_id order. The ordering column should uniquely determine row order for a predictable result.

Q38. How do you find consecutive dates and missing dates in a sequence?

Answer: Use a calendar table or recursive CTE to generate the expected dates, then compare them with the actual dates.

For example, suppose a table called attendance contains an attendance_date column.

```
WITH RECURSIVE calendar AS (
   SELECT DATE('2026-10-01') AS dt
   UNION ALL
   SELECT DATE_ADD(dt, INTERVAL 1 DAY)
   FROM calendar
   WHERE dt < '2026-10-07'
)
SELECT c.dt AS missing_date
FROM calendar c
LEFT JOIN attendance a
   ON a.attendance_date = c.dt
WHERE a.attendance_date IS NULL;
```

This returns dates from October 1 through October 7, 2026, that do not appear in attendance.

To identify consecutive date groups, window functions such as LAG() can compare each date with the preceding date.

Q39. How do you find the frequency of each value in a column?

Answer: Use GROUP BY and COUNT().

```
SELECT department_id,
     COUNT(*) AS frequency
FROM employees
GROUP BY department_id
ORDER BY frequency DESC;
```

This returns the number of employee records for each department ID. If null department IDs exist, they appear as a separate group.

Q40. How do you find employees who joined in the current month or year?

Answer: Use a date range for efficient filtering on an indexed joining-date column.

For the current month:

```
SELECT *
FROM employees
WHERE joining_date >=
    DATE_FORMAT(CURDATE(), '%Y-%m-01')
 AND joining_date <
    DATE_ADD(
       DATE_FORMAT(CURDATE(), '%Y-%m-01'),
       INTERVAL 1 MONTH
    );
```

For the current year:

```
SELECT *
FROM employees
WHERE joining_date >= MAKEDATE(YEAR(CURDATE()), 1)
 AND joining_date <
    MAKEDATE(YEAR(CURDATE()) + 1, 1);
```

These ranges also work when joining_date contains a time component, provided its type and values are compatible with the comparisons.

Q41. How do you find records that occur more than once based on multiple columns?

Answer: Group by the combination of columns and use HAVING COUNT(*) > 1.

```
SELECT emp_name, department_id, salary,
     COUNT(*) AS occurrences
FROM employees
GROUP BY emp_name, department_id, salary
HAVING COUNT(*) > 1;
```

This finds repeated combinations of employee name, department, and salary. It does not necessarily mean the entire records are identical if other columns differ.

Q42. How do you find the percentage contribution of each department to the total salary?

Answer: Divide each department's salary total by the overall salary total and multiply by 100.

```
WITH department_totals AS (
   SELECT department_id,
        SUM(salary) AS dept_salary
   FROM employees
   GROUP BY department_id
)
SELECT department_id,
      dept_salary,
      ROUND(
         100.0 * dept_salary /
         NULLIF(SUM(dept_salary) OVER (), 0),
         2
      ) AS percentage_contribution
FROM department_totals;
```

NULLIF() prevents division by zero. The calculation uses non-null salary values and includes a null department group if one exists.

Q43. How do you paginate query results efficiently?

Answer: For simple pagination, use LIMIT and OFFSET.

```
SELECT emp_id, emp_name, salary
FROM employees
ORDER BY emp_id
LIMIT 10 OFFSET 20;
```

This skips 20 rows and returns up to 10 rows.

For large datasets, keyset pagination is often more efficient because large offsets may require MySQL to scan and discard many rows.

```
SELECT emp_id, emp_name, salary
FROM employees
```

```
WHERE emp_id > 120
ORDER BY emp_id
LIMIT 10;
```

This example assumes emp_id is unique and the last ID on the previous page was 120.

Topic 4: Database Design and Normalization

Q44. What is normalization, and why is it required?

Answer: Normalization is the process of organizing relational tables to reduce unnecessary data duplication and prevent data anomalies.

Its benefits include:

- Improved data consistency.
- Reduced redundant storage.
- Easier maintenance.
- Better enforcement of relationships and dependencies.

For example, storing department details separately from employee records avoids repeatedly storing the same department name for every employee.

Q45. What are insertion, update, and deletion anomalies?

Answer: These are problems caused by poorly structured tables and unnecessary data duplication.

- Insertion anomaly: A fact cannot be inserted without also inserting unrelated data.
- Update anomaly: The same fact must be changed in several places, and some copies may be missed.
- Deletion anomaly: Deleting one record accidentally removes other information that should have been preserved.

Example: If department information exists only in employee rows, deleting the last employee of a department could also remove the only stored record of that department.

Normalization helps prevent these issues.

Q46. What is First Normal Form (1NF)?

Answer: A table is in First Normal Form when each column contains values from its defined domain in a single field and repeating groups or multivalued attributes are not stored as separate repeated columns.

For example, instead of storing phone1, phone2, and phone3 in one student record, phone numbers can be stored in a separate table with one phone number per row.

Important: Atomicity depends on the intended data model; a string containing commas is not automatically a properly modeled multivalued attribute.

Q47. What is Second Normal Form (2NF)?

Answer: A table is in Second Normal Form when it is in 1NF and every non-prime attribute depends on the whole of every candidate key, rather than only part of a composite candidate key.

This primarily addresses partial functional dependencies.

Example: Consider an enrollment table with a composite key of (student_id, course_id) and columns student_name and course_name.

student_name depends only on student_id, and course_name depends only on course_id. These attributes should be moved into their respective Students and Courses tables.

Q48. What is Third Normal Form (3NF)?

Answer: A table is in Third Normal Form when it is in 2NF and has no disallowed transitive dependency of non-prime attributes on candidate keys.

A common practical explanation is that non-key attributes should depend on the key, the whole key, and not on another non-key attribute.

Example: If an employee table stores department_id and department_name, the department name depends on the department ID rather than directly on the employee ID. Department details should generally be stored in a separate Departments table.

The formal definition of 3NF is based on functional dependencies and is more precise than this simplified rule.

Q49. What is the difference between normalization and denormalization?

Answer:

Normalization	**Denormalization**
Organizes data to reduce redundancy	Intentionally introduces redundancy or precomputed data
Helps prevent update anomalies	Can simplify or accelerate selected reads
Often involves splitting data into related tables	May combine data or maintain summaries

Common in transactional systems	Sometimes used in reporting and analytics systems

Normalization is usually a starting point for sound relational design. Denormalization should be based on measured requirements rather than applied automatically.

Q50. What is a functional dependency?

Answer: A functional dependency exists when the value of one attribute or set of attributes determines the value of another attribute or set of attributes.

It is written as:

\[X \rightarrow Y \]

This means that if two rows have the same value for \(X\), they must have the same value for \(Y\).

Example: If student_id uniquely identifies a student, then:

\[student_id \rightarrow student_name \]

This dependency helps identify keys and determine appropriate normalization.

Q51. What is the difference between one-to-one, one-to-many, and many-to-many relationships?

Answer:

- One-to-one: One record in table A corresponds to at most one record in table B, and vice versa.
- One-to-many: One record in table A can relate to multiple records in table B.
- Many-to-many: Multiple records in each table can relate to multiple records in the other table.

Examples:

- One-to-one: A user and a unique user profile.
- One-to-many: A department and its employees.
- Many-to-many: Students and courses, connected through enrollments.

Foreign keys and appropriate uniqueness constraints enforce the intended relationship rules.

Q52. How do you convert an ER diagram into relational tables?

Answer: Converting an Entity-Relationship diagram into relational tables typically involves these steps:

1. Create a table for each entity.
2. Choose a primary key for each table.
3. Convert attributes into columns with suitable data types.
4. Represent one-to-many relationships using a foreign key on the many side.
5. Represent many-to-many relationships using a junction table.
6. Represent one-to-one relationships using a foreign key with a suitable unique constraint.
7. Apply NOT NULL, UNIQUE, CHECK, and other constraints according to business rules.

Example: An ER diagram containing students, courses, and their many-to-many enrollment relationship becomes three tables: Students, Courses, and Enrollments.

Q53. What is the difference between a natural key and a surrogate key when designing database tables?

Answer:

- Natural key: Uses an attribute with business meaning, such as an official employee number.
- Surrogate key: Uses an artificial identifier without inherent business meaning, such as an auto-incremented integer.

Example:

```
CREATE TABLE employees (
  emp_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  employee_code VARCHAR(30) NOT NULL UNIQUE,
  emp_name VARCHAR(100) NOT NULL
);
```

Here, emp_id is a surrogate key, while employee_code is a candidate natural key.

Surrogate keys are often stable and convenient for relationships, but natural-key uniqueness should still be enforced when required by the business.

Q54. How would you design a student management database with students, courses, and enrollments?

Answer: Use separate tables for students and courses, and a junction table for enrollments.

```
CREATE TABLE students (
```

```
    student_id INT PRIMARY KEY,
    student_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE
);

CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL
);

CREATE TABLE enrollments (
    student_id INT,
    course_id INT,
    enrollment_date DATE NOT NULL,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id)
        REFERENCES students(student_id),
    FOREIGN KEY (course_id)
        REFERENCES courses(course_id)
);
```

This design supports multiple courses per student and multiple students per course while avoiding duplicated student and course details.

Q55. When would you intentionally denormalize a database?

Answer: Denormalization may be appropriate when a read-heavy workload needs faster access to frequently requested information and the performance benefit justifies the added complexity.

Examples include:

- Storing precomputed totals for reporting.
- Maintaining summary tables for dashboards.

- Duplicating selected attributes to reduce expensive joins.
- Building read models for analytical workloads.

The trade-offs include extra storage, more complicated writes, and the risk of inconsistent copies. Denormalization should follow performance measurement and a plan for maintaining consistency.

Topic 5: Views

Q56. What is a view in MySQL?

Answer: A view is a named SQL query that can be used like a virtual table. A regular view stores its query definition rather than a separate persistent copy of the result rows.

Example:

```
CREATE VIEW high_salary_employees AS

SELECT emp_id, emp_name, salary

FROM employees

WHERE salary > 70000;
```

Query the view:

```
SELECT *

FROM high_salary_employees;
```

Q57. What is the difference between a view and a table?

Answer:

View	**Table**
Usually stores a query definition	Stores table data
Retrieves results from its underlying objects	Stores rows directly
Can simplify complex queries	Stores the underlying records
May be updatable depending on its definition	Supports data modification subject to constraints
Does not ordinarily store its own result rows	Stores persistent data

A regular view does not automatically improve performance. Its query may still need to access the underlying tables.

Q58. How do you create, replace, and drop a view?

Answer: Use CREATE VIEW to create a view, CREATE OR REPLACE VIEW to replace its definition, and DROP VIEW to remove it.

Create:

```
CREATE VIEW employee_details AS

SELECT emp_id, emp_name, department_id

FROM employees;
```

Replace:

```
CREATE OR REPLACE VIEW employee_details AS

SELECT emp_id, emp_name, salary

FROM employees;
```

Drop:

```
DROP VIEW employee_details;
```

Dropping a view removes the view definition, not the underlying employee records.

Q59. Can we insert, update, or delete records through a view?

Answer: Yes, if the view is updatable and the operation satisfies the relevant constraints.

For example, a simple view referencing a single table without disqualifying constructs may allow updates:

```
CREATE VIEW employee_names AS

SELECT emp_id, emp_name

FROM employees;
```

An update through the view:

```
UPDATE employee_names

SET emp_name = 'Ravi Kumar'

WHERE emp_id = 101;
```

This updates the underlying employee record, assuming the operation is permitted.

Q60. What makes a view updatable or non-updatable?

Answer: A view is updatable when MySQL can map the requested change unambiguously to the underlying table rows and columns.

Simple views based on a single table are often updatable.

Views involving constructs such as GROUP BY, aggregate functions, DISTINCT, UNION, and certain joins or derived tables are generally not updatable in the ordinary sense.

Example of a non-updatable aggregate view:

```
CREATE VIEW department_totals AS
SELECT department_id, COUNT(*) AS total_employees
FROM employees
GROUP BY department_id;
```

The aggregated count cannot be directly updated as though it were an employee column.

Q61. Can a view be created using multiple tables?

Answer: Yes. A view can combine data from multiple tables using joins.

```
CREATE VIEW employee_department_view AS
SELECT e.emp_id,
    e.emp_name,
    d.department_name
FROM employees e
JOIN departments d
  ON e.department_id = d.department_id;
```

This view provides employee and department details through one queryable interface.

Whether a view involving multiple tables can be updated depends on its definition and the operation being attempted.

Q62. Can one view be created using another view?

Answer: Yes. A view can reference another view, provided the referenced objects exist and the resulting definition is valid.

```
CREATE VIEW high_salary_view AS
SELECT emp_id, emp_name, salary
FROM employees
WHERE salary > 70000;
```

```
CREATE VIEW high_salary_names AS

SELECT emp_id, emp_name

FROM high_salary_view;
```

This allows database queries to be organized into reusable layers. However, excessive view nesting can make dependencies and query performance harder to understand.

Q63. What are the advantages and limitations of views?

Answer:

Advantages:

- Simplify complex SQL queries.
- Present selected columns to users.
- Support logical data abstraction.
- Help standardize commonly used queries.
- Can contribute to access control when combined with appropriate privileges.

Limitations:

- Regular views do not ordinarily store their own result rows.
- Complex definitions can make queries slower or harder to optimize.
- Some views cannot be updated.
- Changes to underlying tables may affect dependent views.
- Views alone do not replace a complete security design.

Q64. How can views restrict access to sensitive columns or rows?

Answer: Create a view that exposes only the permitted rows and columns, then grant the relevant users access to the view rather than the underlying table.

Example:

```
CREATE VIEW public_employee_details AS

SELECT emp_id, emp_name, department_id

FROM employees;
```

A user can be granted permission to query the view:

```
GRANT SELECT

ON company.public_employee_details
```

TO 'report_user'@'localhost';

This assumes the account exists and that the executing administrator has the required privileges.

For effective restriction, avoid granting the user broader privileges on the underlying table that would bypass the view.

Q65. What is the difference between a regular view and a materialized view?

Answer:

A regular MySQL view stores a query definition and retrieves its results from the underlying objects when queried.

A materialized view stores the result of a query physically and must be refreshed or otherwise maintained when source data changes.

Important: MySQL does not provide native materialized views as a standard database object. Similar behavior can be implemented using summary tables, scheduled jobs, or application-managed refresh logic.

Topic 6: Indexes — Fundamentals

Q66. What is an index in MySQL, and why is it used?

Answer: An index is a data structure that helps MySQL locate rows more efficiently without scanning every row in a table.

Indexes can improve:

- Filtering with WHERE.
- Table joins.
- Sorting and grouping in suitable cases.
- Enforcement of primary key and unique constraints.

Example:

CREATE INDEX idx_employee_name

ON employees(emp_name);

This creates an index on emp_name, which may improve queries that search or sort by that column.

Q67. What are the different types of indexes available in MySQL?

Answer: Common index types and categories include:

- Primary key index: Uniquely identifies table rows.
- Unique index: Enforces uniqueness.
- Non-unique index: Supports lookups without enforcing uniqueness.
- Composite index: Covers multiple columns.
- Full-text index: Supports full-text searching on supported text columns.
- Spatial index: Supports spatial data operations for supported types and storage engines.
- B-tree indexes: The standard index structure for many InnoDB indexes.
- Hash indexes: Available in specific contexts, such as the MEMORY storage engine and certain internal InnoDB structures.

These categories overlap. For example, a unique index on two columns is also a composite index.

Q68. What is the difference between a primary key index and a secondary index?

Answer: In InnoDB, the primary key is stored in the clustered index, which organizes table rows by primary key.

A secondary index stores its indexed values along with the corresponding primary key value.

Primary key index	**Secondary index**
Organizes the clustered table data	Provides an additional access path
Must be unique and non-null	Can be unique or non-unique
One primary key per table	Multiple secondary indexes can exist
Primary key values identify clustered rows	InnoDB secondary entries contain the primary key

A lookup through a secondary index may require an additional lookup into the clustered index unless the secondary index covers the query.

Q69. What is the difference between a unique index and a non-unique index?

Answer:

- Unique index: Prevents duplicate indexed values or duplicate indexed combinations.
- Non-unique index: Allows duplicate values.

Example:

```
CREATE UNIQUE INDEX idx_employee_email
ON employees(email);

CREATE INDEX idx_employee_department
ON employees(department_id);
```

The first enforces unique non-null email values, while the second allows multiple employees to belong to the same department.

In MySQL, a unique index generally permits multiple NULL values.

Q70. What is a composite index?

Answer: A composite index is an index defined on two or more columns.

Example:

```
CREATE INDEX idx_dept_salary
ON employees(department_id, salary);
```

This index may help queries filtering by department_id, or by both department_id and salary.

The column order matters because a composite index is ordered by its leading columns first.

Q71. What is the leftmost-prefix rule?

Answer: The leftmost-prefix rule means that a composite B-tree index can generally be used to support lookups involving its leading column or a leading sequence of columns.

Given:

```
CREATE INDEX idx_dept_salary_name
ON employees(department_id, salary, emp_name);
```

The index can generally support queries using:

- department_id
- department_id and salary
- department_id, salary, and emp_name

It generally cannot efficiently use the index as a direct lookup path for salary alone, because the leading column is omitted.

Actual index usage depends on the query, available statistics, and optimizer decisions.

Q72. What is a covering index?

Answer: A covering index contains all the columns needed by a particular query, allowing MySQL to retrieve the required data from the index without an additional table-row lookup.

Example:

CREATE INDEX idx_dept_name

ON employees(department_id, emp_name);

Query:

SELECT emp_name

FROM employees

WHERE department_id = 10;

The index contains both the filtering column and the selected column, so it may cover this query.

A covering index can improve performance, but unnecessarily wide indexes increase storage requirements and write overhead.

Q73. What is the difference between a full table scan and an index scan?

Answer:

- Full table scan: MySQL reads the table's rows to evaluate the query.
- Index scan or lookup: MySQL reads an index to locate matching entries, potentially retrieving only a subset of rows.

An index is often useful for selective conditions. A full table scan may be faster when a large proportion of the table must be read.

Interview point: A full table scan is not always a problem. The correct execution plan depends on data size, selectivity, row width, and the query's requirements.

Q74. What are the disadvantages of creating too many indexes?

Answer: Too many indexes can:

1. Consume additional disk space.
2. Slow down inserts because indexes must be maintained.
3. Slow down updates when indexed columns change.
4. Slow down deletes because corresponding index entries must be removed.
5. Increase maintenance and backup overhead.

6. Make index selection and schema management more complicated.

Indexes should be created based on actual query patterns and validated with execution plans and performance measurements.

Q75. How do you create, inspect, and drop indexes in MySQL?

Answer:

Create an index:

```
CREATE INDEX idx_emp_salary
ON employees(salary);
```

Inspect indexes:

```
SHOW INDEX FROM employees;
```

Drop a secondary index:

```
DROP INDEX idx_emp_salary
ON employees;
```

For a primary key, use a different statement:

```
ALTER TABLE employees
DROP PRIMARY KEY;
```

Dropping a primary key may fail if foreign keys depend on it or if the table definition requires changes. It should not be done casually.

Q76. What is the difference between a B-tree index and a hash index?

Answer:

B-tree index	**Hash index**
Supports equality and range lookups	Primarily supports equality lookups
Supports ordered traversal	Does not provide ordered traversal
Can help with range conditions and suitable sorting	Not suitable for range scans
Used for many standard InnoDB indexes	Available in specific MySQL contexts

InnoDB uses B-tree indexes for ordinary table indexes. The MEMORY engine supports hash indexes by default for applicable indexes and also supports B-tree indexes.

Q77. How do indexes affect INSERT, UPDATE, and DELETE performance?

Answer: Indexes improve many read operations but create additional work during data modifications.

- INSERT: New entries must be added to relevant indexes.
- UPDATE: Index entries may need modification when indexed values change.
- DELETE: Corresponding entries must be removed from indexes.

Real-time example: An employee table with many unnecessary indexes may perform slower inserts and updates than a table with a carefully selected set of indexes.

The objective is to balance read performance against write overhead and storage cost.

Topic 7: Transactions and Basic Concurrency

Q78. What is a transaction in MySQL?

Answer: A transaction is a sequence of database operations treated as one logical unit of work.

A transaction is committed when its changes are accepted, or rolled back when its changes need to be undone.

Example:

```
START TRANSACTION;

UPDATE accounts
SET balance = balance - 1000
WHERE account_id = 101;

UPDATE accounts
SET balance = balance + 1000
WHERE account_id = 102;

COMMIT;
```

This transfers 1,000 from one account to another. In a real banking application, validation must ensure that the source account exists and has sufficient funds before the transfer is committed.

Q79. What are the ACID properties?

Answer: ACID describes four properties of reliable transactions.

- Atomicity: All required operations succeed together, or the transaction is rolled back.
- Consistency: A transaction preserves the database's defined integrity rules.
- Isolation: Concurrent transactions are controlled so that their interactions follow the selected isolation level.
- Durability: Once committed, changes survive failures according to the database's durability guarantees.

Real-time example: During a bank transfer, money should not be deducted from one account without the corresponding credit to the other account being committed.

Q80. What is the difference between COMMIT, ROLLBACK, and SAVEPOINT?

Answer:

Command	Purpose
COMMIT	Permanently commits the current transaction's changes
ROLLBACK	Undoes uncommitted changes in the current transaction
SAVEPOINT	Marks a point within a transaction for a possible partial rollback

Example:

```
START TRANSACTION;

UPDATE accounts
SET balance = balance - 500
WHERE account_id = 101;

SAVEPOINT after_debit;

UPDATE accounts
```

```
SET balance = balance + 500

WHERE account_id = 102;


ROLLBACK TO SAVEPOINT after_debit;


COMMIT;
```

This example deliberately rolls back the credit operation but commits the debit, so it is not a valid money-transfer implementation. It demonstrates savepoint behavior only.

For a transfer, both balance changes must succeed together, or the whole transaction must be rolled back.

Q81. What is autocommit, and how does it affect transactions?

Answer: Autocommit is a session setting that determines whether each statement is committed automatically when it completes successfully.

In MySQL, autocommit is enabled by default.

Check the setting:

```
SELECT @@autocommit;
```

Disable it for the current session:

```
SET autocommit = 0;
```

When autocommit is disabled, changes can remain uncommitted until COMMIT or ROLLBACK is issued. Explicit transactions using START TRANSACTION are generally clearer when several operations must succeed together.

Important: DDL statements such as CREATE TABLE, ALTER TABLE, and TRUNCATE TABLE can cause implicit commits in MySQL, so they should not be assumed to behave like ordinary transactional DML.

Q82. What is the difference between transactional and non-transactional storage engines?

Answer:

A transactional storage engine supports transaction operations such as COMMIT and ROLLBACK for its data changes. A non-transactional engine does not provide the same rollback guarantees.

Transactional engine	**Non-transactional engine**

Supports transactions	Does not provide ordinary transactional rollback
Supports rollback of applicable uncommitted changes	Changes may remain even when a rollback is requested
Supports transaction isolation features	Does not provide equivalent transactional isolation
Example: InnoDB	Example: MyISAM

Interview point: InnoDB is the usual choice for applications requiring transactions, foreign keys, and reliable concurrent updates.

Q83. What is a deadlock, and why can it occur during concurrent transactions?

Answer: A deadlock occurs when two or more transactions wait for resources held by one another, creating a cycle that prevents them from progressing.

Example:

- Transaction A locks row 1 and then requests row 2.
- Transaction B locks row 2 and then requests row 1.
- Each transaction waits for the other to release its lock.

InnoDB detects deadlocks and normally rolls back one of the transactions to break the cycle.

Ways to reduce deadlocks:

1. Access records in a consistent order.
2. Keep transactions short.
3. Use appropriate indexes to reduce the number of rows that must be examined and locked.
4. Handle deadlock errors by retrying the entire transaction when appropriate.

Deadlocks are possible even in correctly designed transactional applications.

Q84. What happens if a transaction fails before it is committed?

Answer: The outcome depends on the type of failure and the transaction's state.

For an InnoDB transaction, a statement error may roll back only the failed statement, leaving the transaction active. Certain errors, including deadlocks, can cause the entire transaction to be rolled back. A connection failure can also leave the transaction uncommitted, and InnoDB recovery will handle incomplete transactions.

Example:

START TRANSACTION;

UPDATE accounts

SET balance = balance - 500

WHERE account_id = 101;

ROLLBACK;

The uncommitted update is undone.

Important: Do not assume every SQL error automatically rolls back the entire transaction. Application code should handle errors and explicitly roll back when the business operation cannot safely continue.

Q85. How do you transfer money between two bank accounts while maintaining transaction consistency?

Answer: Use a transaction so that the debit and credit are committed together. Validate the source account, verify sufficient funds, and ensure the destination account exists.

A simplified MySQL example is:

START TRANSACTION;

UPDATE accounts

SET balance = balance - 1000

WHERE account_id = 101

  AND balance >= 1000;

-- Application must verify exactly one row was updated.

-- If not, ROLLBACK and report failure.

UPDATE accounts

SET balance = balance + 1000

```
WHERE account_id = 102;


-- Application must verify exactly one row was updated.

-- If not, ROLLBACK the entire transaction.


COMMIT;
```

How it works:

1. The transaction begins.
2. The source account is debited only if sufficient funds are available.
3. The application checks the affected-row count.
4. The destination account is credited.
5. The application checks that the destination update succeeded.
6. If either operation fails validation, the application rolls back the entire transaction; otherwise, it commits.

Important: The example describes the transaction flow, but the comments require actual application-side checks. If the destination update fails, the application must issue ROLLBACK rather than continue to COMMIT. Production implementations should also validate distinct and existing account IDs, handle concurrent transfers and deadlocks, and use appropriate isolation and authorization controls.

The ACID properties help ensure that the debit and credit are treated as one logical operation rather than two independent updates.

Advaned Level

==========

Topic 1: Stored Procedures and Stored Functions

Q1. What is a stored procedure, and why is it used?

Answer: A stored procedure is a named collection of SQL statements stored in the database and executed using the CALL statement.

It is used to encapsulate business logic, perform repeated database operations, reduce repeated SQL in applications, and control access to database operations.

Example:

```
DELIMITER //


CREATE PROCEDURE GetAllEmployees()

BEGIN

   SELECT * FROM employees;

END //


DELIMITER ;
```

Execute it using:

```
CALL GetAllEmployees();
```

Real-time scenario: A payroll application can use a stored procedure to calculate monthly salary details and retrieve employee payroll records.

Q2. What is the difference between a stored procedure and a stored function?

Answer:

Stored procedure	**Stored function**
Invoked using CALL	Invoked as part of an expression
Can return result sets and use output parameters	Returns a single value through RETURN
Can use IN, OUT, and INOUT parameters	Has input parameters; OUT and INOUT parameters are not supported
Can manage transactions in permitted contexts	Cannot independently commit or roll back transactions

Useful for multi-step operations Useful for calculations and reusable expressions

Example function:

```
CREATE FUNCTION CalculateBonus(salary DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
RETURN salary * 0.10;
```

Use it as:

```
SELECT CalculateBonus(50000);
```

The result is 5000.00.

Q3. What is the difference between IN, OUT, and INOUT parameters?

Answer:

- IN: Supplies a value to the procedure. It is the default parameter mode.
- OUT: Returns a value from the procedure to the caller.
- INOUT: Supplies an initial value and returns a modified value.

Example:

```
DELIMITER //

CREATE PROCEDURE CalculateTotal(
   IN price DECIMAL(10,2),
   IN quantity INT,
   OUT total DECIMAL(10,2)
)
BEGIN
   SET total = price * quantity;
END //

DELIMITER ;
```

Call the procedure:

```
CALL CalculateTotal(500, 3, @total);


SELECT @total;
```

Output: 1500.00.

Real-time scenario: An invoice procedure receives a unit price and quantity and returns the calculated invoice amount.

Q4. What is the purpose of the DELIMITER command when creating stored procedures?

Answer: DELIMITER changes the statement terminator used by the MySQL command-line client.

Stored procedures often contain multiple SQL statements ending in semicolons. Changing the delimiter temporarily prevents the client from treating each internal semicolon as the end of the complete procedure definition.

Example:

```
DELIMITER //


CREATE PROCEDURE ShowMessage()
BEGIN
   SELECT 'Hello';
   SELECT 'Welcome to MySQL';
END //


DELIMITER ;
```

Important: DELIMITER is a client command, not an SQL statement executed by the MySQL server. GUI tools and database APIs may use different ways to submit procedure definitions.

Q5. Can one stored procedure call another stored procedure?

Answer: Yes. A stored procedure can invoke another stored procedure using CALL.

Example:

```
DELIMITER //
```

```
CREATE PROCEDURE GetEmployeeCount(
    OUT total INT
)
BEGIN
    SELECT COUNT(*) INTO total
    FROM employees;
END //

CREATE PROCEDURE DisplayEmployeeCount()
BEGIN
    CALL GetEmployeeCount(@count);
    SELECT @count AS total_employees;
END //

DELIMITER ;
```

Execute it:

```
CALL DisplayEmployeeCount();
```

This allows database operations to be divided into reusable procedures.

Q6. How do DECLARE HANDLER, CONTINUE, and EXIT handlers work?

Answer: A handler specifies what a stored program should do when a particular SQL condition occurs.

- CONTINUE handler: Executes its handler action and continues execution after the statement that raised the condition.
- EXIT handler: Executes its handler action and exits the enclosing BEGIN ... END block in which the handler was declared.
- DECLARE HANDLER: Declares the handler and the condition it handles.

Example:

```
DELIMITER //
```

```
CREATE PROCEDURE GetEmployee(
   IN p_id INT
)
BEGIN
   DECLARE CONTINUE HANDLER FOR NOT FOUND
      SELECT 'Employee not found' AS message;

   SELECT emp_name
   FROM employees
   WHERE emp_id = p_id;
END //

DELIMITER ;
```

Important: NOT FOUND is commonly used with cursors and SELECT ... INTO statements that return no rows. A handler's behavior depends on its declared condition and action.

Q7. What is the difference between SIGNAL and RESIGNAL?

Answer:

- SIGNAL: Raises a specified SQL condition or error.
- RESIGNAL: Rethrows the current condition or raises it again with modified condition information, typically inside a handler.

Example using SIGNAL:

```
IF p_amount <= 0 THEN
   SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Amount must be positive';
END IF;
```

Example using RESIGNAL:

```
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
   ROLLBACK;
```

```
    RESIGNAL;
END;
```

The second example rolls back the transaction and propagates the original SQL exception.

Real-time scenario: A payment procedure can reject invalid payment amounts with SIGNAL and propagate unexpected database errors with RESIGNAL.

Q8. How do transactions and error handling work inside stored procedures?

Answer: A stored procedure can execute transaction statements and declare handlers to respond to SQL errors.

For a transactional workflow, the procedure can begin a transaction, execute the required statements, commit on success, and roll back when an exception occurs.

Example:

```
DELIMITER //

CREATE PROCEDURE UpdateEmployeeSalary(
    IN p_id INT,
    IN p_salary DECIMAL(10,2)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    UPDATE employees
    SET salary = p_salary
    WHERE emp_id = p_id;
```

```
    COMMIT;
END //

DELIMITER ;
```

Important: This is a simplified example. Production code should validate the salary, verify the affected-row count, and consider whether the procedure should own the transaction or participate in a transaction managed by its caller. MySQL does not provide independently nestable transactions through nested START TRANSACTION statements.

Q9. How do cursors and loops work inside stored procedures?

Answer: A cursor processes a query result one row at a time. A loop repeatedly executes statements until a specified condition is met.

Cursors are useful when a task genuinely requires row-by-row processing, although set-based SQL is usually preferable for bulk operations.

Example:

```
DELIMITER //

CREATE PROCEDURE DisplayEmployeeNames()
BEGIN
    DECLARE finished INT DEFAULT 0;
    DECLARE v_name VARCHAR(100);

    DECLARE emp_cursor CURSOR FOR
        SELECT emp_name FROM employees;

    DECLARE CONTINUE HANDLER FOR NOT FOUND
        SET finished = 1;

    OPEN emp_cursor;
```

```
  read_loop: LOOP
    FETCH emp_cursor INTO v_name;

    IF finished = 1 THEN
      LEAVE read_loop;
    END IF;

    SELECT v_name AS employee_name;
  END LOOP;

  CLOSE emp_cursor;
END //

DELIMITER ;
```

Important: The cursor, variables, and handlers must be declared in the correct order. In this example, the cursor handler is intended to handle the end of the cursor result set.

Q10. What are the advantages, limitations, and security considerations of stored procedures?

Answer:

Advantages:

- Reusable database logic.
- Reduced duplication of SQL.
- Fewer application-to-database round trips for some workflows.
- Centralized validation and transaction handling.
- Ability to grant execution privileges without granting broad direct table access.

Limitations:

- Database-specific syntax can reduce portability.
- Complex procedures can be difficult to test and maintain.

- Debugging may be less convenient than application-level debugging.
- Excessive row-by-row processing can hurt performance.

Security considerations:

- Use parameterized procedure calls and avoid unsafe dynamic SQL.
- Apply least-privilege permissions.
- Understand SQL SECURITY DEFINER versus SQL SECURITY INVOKER.
- Protect sensitive data in logs and error messages.
- Validate input and enforce business rules consistently.

Q11. How would you design a stored procedure to transfer money safely between two bank accounts?

Answer: The procedure should debit the source account and credit the destination account within the same transaction. It must validate the amount, verify both accounts, prevent insufficient-balance withdrawals, and roll back if either operation fails.

A simplified example for an InnoDB accounts table follows:

```
DELIMITER //

CREATE PROCEDURE TransferMoney(
  IN p_from INT,
  IN p_to INT,
  IN p_amount DECIMAL(12,2)
)
BEGIN
  DECLARE v_rows INT;

  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;
```

```
IF p_amount <= 0 OR p_from = p_to THEN
   SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Invalid transfer';
END IF;

START TRANSACTION;

UPDATE accounts
SET balance = balance - p_amount
WHERE account_id = p_from
  AND balance >= p_amount;

SET v_rows = ROW_COUNT();

IF v_rows <> 1 THEN
   SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT =
         'Source account missing or insufficient funds';
END IF;

UPDATE accounts
SET balance = balance + p_amount
WHERE account_id = p_to;

SET v_rows = ROW_COUNT();

IF v_rows <> 1 THEN
```

```
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Destination account not found';
    END IF;

    COMMIT;
END //

DELIMITER ;
```

How it works:

1. Rejects invalid transfer requests.
2. Begins a transaction.
3. Debits the source only if sufficient funds are available.
4. Checks that exactly one source row was updated.
5. Credits the destination and checks the affected-row count.
6. Commits only if both updates succeed.
7. Rolls back and propagates errors through the exception handler.

Production considerations: Use InnoDB, appropriate authorization, transaction monitoring, and a strategy for deadlocks and retries. For real financial systems, also consider idempotency, audit records, currency handling, and a reliable ledger design.

Topic 2: Triggers and Database Automation

Q12. What is a trigger in MySQL, and why is it used?

Answer: A trigger is a database object that automatically executes a predefined block of SQL statements when a specified event occurs on a table.

Triggers are commonly used for auditing, validation, and maintaining related data.

Example:

```
CREATE TRIGGER before_employee_insert
BEFORE INSERT ON employees
FOR EACH ROW
SET NEW.emp_name = TRIM(NEW.emp_name);
```

This trigger removes leading and trailing spaces from an employee's name before insertion.

Real-time scenario: A trigger can record salary changes in an audit table whenever an employee's salary is updated.

Q13. What are the different types of triggers supported by MySQL?

Answer: MySQL supports triggers based on two dimensions: timing and event.

Timing:

- BEFORE — executes before the triggering operation.
- AFTER — executes after the triggering operation.

Events:

- INSERT
- UPDATE
- DELETE

These combinations produce six basic trigger types: BEFORE INSERT, AFTER INSERT, BEFORE UPDATE, AFTER UPDATE, BEFORE DELETE, and AFTER DELETE.

MySQL triggers execute for each affected row; MySQL does not support statement-level triggers.

Q14. What is the difference between BEFORE and AFTER triggers?

Answer:

BEFORE trigger	**AFTER trigger**
Executes before the row operation completes	Executes after the row operation succeeds
Can modify NEW values in applicable INSERT and UPDATE triggers	Cannot modify NEW values
Useful for validation and adjusting values	Useful for auditing and related actions

Example:

```
CREATE TRIGGER before_salary_update
BEFORE UPDATE ON employees
FOR EACH ROW
```

SET NEW.salary = ROUND(NEW.salary, 2);

This rounds the new salary value before the update is applied.

Q15. What are OLD and NEW in MySQL triggers?

Answer: OLD and NEW are trigger references used to access row values.

- OLD represents the existing row values.
- NEW represents the incoming or updated row values.

Their availability depends on the event.

Event	OLD	NEW
INSERT	Not available	Available
UPDATE	Available	Available
DELETE	Available	Not available

Example:

```
IF NEW.salary < 0 THEN
   SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Salary cannot be negative';
END IF;
```

This validation can be used inside a BEFORE INSERT or BEFORE UPDATE trigger.

Q16. Can multiple triggers exist for the same event and timing?

Answer: Yes. Modern MySQL versions allow multiple triggers for the same table, event, and timing.

For example, a table can have multiple BEFORE UPDATE triggers. Their execution order can be specified using FOLLOWS or PRECEDES when creating triggers, relative to another trigger.

Real-time scenario: One trigger can validate salary values, while another prepares audit-related values before an employee update.

Important: Keep trigger behavior simple and predictable. Multiple triggers can create hidden dependencies that make debugging harder.

Q17. Can a trigger invoke a stored procedure?

Answer: A trigger can invoke a stored procedure using CALL, subject to MySQL's restrictions on statements that stored procedures and triggers can execute.

However, a procedure called from a trigger cannot return a result set to the client or use prohibited transaction-control statements such as COMMIT and ROLLBACK.

Real-time scenario: An employee trigger may call a procedure that performs a permitted audit-related database operation.

Best practice: Keep trigger logic lightweight and avoid complex chains of triggers and procedures.

Q18. Can a trigger modify the same table that caused it to execute?

Answer: A trigger generally cannot modify the table that is already being used by the statement that activated the trigger. MySQL commonly reports error 1442 when a trigger or stored function attempts to modify such a table.

For a BEFORE INSERT or BEFORE UPDATE trigger, modify NEW.column_name directly when the operation permits it rather than issuing another UPDATE against the same table.

Example: To normalize an employee's name, assign a cleaned value to NEW.emp_name in a BEFORE INSERT trigger.

Q19. How do triggers interact with transactions in InnoDB?

Answer: Changes made by triggers operating on transactional InnoDB tables participate in the transaction that caused the trigger to execute.

If the transaction is rolled back, the trigger's transactional changes are rolled back as well.

Real-time scenario: If an employee salary update inserts an audit record through an InnoDB trigger, rolling back the salary update also rolls back that audit insertion.

Important: Avoid mixing transactional and non-transactional storage engines when atomic behavior is required. A rollback cannot undo changes already made to a non-transactional table.

Q20. What are the limitations and performance risks of triggers?

Answer:

Common limitations and risks include:

- Triggers execute automatically, so their behavior may not be obvious to application developers.
- Row-level execution can add substantial overhead during bulk updates or inserts.
- Complex trigger chains are difficult to debug.
- MySQL restricts certain SQL statements inside triggers.
- Triggers cannot return ordinary result sets to the calling client.

- Incorrect trigger logic can cause unexpected errors or transaction rollbacks.

Best practice: Use triggers for small, database-enforced tasks such as auditing or normalization. Keep complex business workflows in an appropriate service or stored procedure where their behavior can be tested and monitored.

Q21. How would you design an audit trigger to track changes to employee salary records?

Answer: Create an audit table to store the employee ID, previous salary, new salary, change timestamp, and database user responsible for the change.

```
CREATE TABLE salary_audit (
   audit_id BIGINT AUTO_INCREMENT PRIMARY KEY,
   emp_id INT NOT NULL,
   old_salary DECIMAL(10,2),
   new_salary DECIMAL(10,2),
   changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   changed_by VARCHAR(288)
);
```

Then create an AFTER UPDATE trigger:

```
DELIMITER //

CREATE TRIGGER audit_salary_change
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
   IF NOT (OLD.salary <=> NEW.salary) THEN
      INSERT INTO salary_audit (
         emp_id,
         old_salary,
         new_salary,
         changed_by
      )
```

```
        VALUES (
            NEW.emp_id,
            OLD.salary,
            NEW.salary,
            USER()
        );
    END IF;
END //

DELIMITER ;
```

The <=> operator is MySQL's null-safe equality operator. Negating it detects salary changes even when one value is NULL.

Real-time scenario: When an employee's salary changes, the audit table records the previous and new values.

Important: USER() identifies the connection's user and host, which may not be the application's actual end user. For reliable user-level auditing, the application should establish a trusted identity context.

Topic 3: Advanced Indexing and Query Optimization

Q22. How does the MySQL query optimizer choose an execution plan?

Answer: The MySQL optimizer evaluates alternative ways to execute a query and selects a plan based on estimated cost.

It considers factors such as:

- Available indexes.
- Table and index statistics.
- Estimated rows and selectivity.
- Join order.
- Join algorithms and access methods.
- Sorting, grouping, and filtering requirements.
- Estimated I/O and processing costs.

Example: For a query filtering employees by a highly selective indexed employee ID, the optimizer may choose an index lookup instead of a full table scan.

The selected plan is based on estimates, so inaccurate statistics or incorrect cardinality estimates can lead to poor performance.

Q23. How do you analyze a query using EXPLAIN and EXPLAIN ANALYZE?

Answer: EXPLAIN shows the optimizer's planned execution strategy. EXPLAIN ANALYZE executes the query and reports the actual execution details alongside estimates.

Example:

```
EXPLAIN

SELECT *

FROM employees

WHERE department_id = 10;
```

For actual execution analysis in supported MySQL versions:

```
EXPLAIN ANALYZE

SELECT *

FROM employees

WHERE department_id = 10;
```

Important details to examine include:

- Access method.
- Selected index.
- Estimated rows.
- Actual rows and loops in EXPLAIN ANALYZE.
- Filtering and join operations.
- Actual execution time compared with estimated costs.

Important: EXPLAIN ANALYZE executes the statement. Use it carefully with expensive queries, and do not casually apply it to statements that modify production data.

Q24. Why might MySQL ignore an existing index?

Answer: MySQL may choose not to use an index when the optimizer estimates that another execution plan is cheaper.

Common reasons include:

1. The query returns a large proportion of the table.
2. The indexed column has low selectivity.
3. The query applies a function to the indexed column.
4. An implicit type conversion interferes with index usage.
5. The composite index's leading columns are not used effectively.
6. The index does not cover the required filtering or ordering.
7. Statistics are outdated or inaccurate.

Example: A query retrieving nearly every row may be faster with a table scan than with an index followed by many individual row lookups.

Use EXPLAIN to investigate the optimizer's choice rather than assuming every index must be used.

Q25. How does InnoDB implement clustered and secondary indexes?

Answer: InnoDB stores table rows in a clustered index organized by the primary key.

- Clustered index: Contains the table's row data organized by primary key.
- Secondary index: Contains indexed column values along with the primary key value used to locate the clustered row.

If a query uses a secondary index but needs columns not stored in that index, InnoDB may perform an additional clustered-index lookup.

Real-time scenario: An index on department_id can locate employees in a department, while the primary key identifies the corresponding employee rows.

Interview point: Keep primary keys reasonably compact because their values are also stored in InnoDB secondary index entries.

Q26. How does the order of columns in a composite index affect query performance?

Answer: The order determines which leading-column combinations can efficiently use the index.

Consider:

CREATE INDEX idx_dept_salary

ON employees(department_id, salary);

This is generally useful for queries filtering by department_id alone or by both department_id and salary.

It is not usually the ideal index for a query filtering only by salary.

Example:

```
SELECT *
FROM employees
WHERE department_id = 10
  AND salary > 70000;
```

The composite index can support the department lookup and salary range.

Choose column order based on actual query patterns, filtering conditions, sorting requirements, and selectivity.

Q27. What is index selectivity, and why does it matter?

Answer: Index selectivity describes how effectively an index distinguishes different values and narrows the number of matching rows.

A column with many distinct values, such as a unique employee ID, generally has higher selectivity than a column with only a few distinct values, such as a yes/no status.

A selective index is often more useful for queries that return relatively few rows.

Example: An index on a unique order ID is generally effective for finding one order, while an index on a column where most rows have the same status may be less useful for some status-based queries.

Selectivity is important, but it is not the only factor in the optimizer's decision.

Q28. What is index condition pushdown?

Answer: Index Condition Pushdown (ICP) is an optimization in which MySQL evaluates suitable conditions using index entries before fetching the corresponding full table rows.

This can reduce unnecessary row lookups.

Example: With a composite index on (department_id, emp_name), a query filtering by department and a suitable condition on employee name may allow some filtering to occur within the storage engine.

ICP is supported for applicable access methods and storage engines, including InnoDB. Whether it is used depends on the query and execution plan.

Q29. How do functions and implicit type conversions on indexed columns affect index usage?

Answer: Applying a function or causing an implicit conversion on an indexed column can make it harder for MySQL to use the index as a direct lookup path.

Example:

```
SELECT *
FROM employees
WHERE YEAR(joining_date) = 2026;
```

A more index-friendly alternative is:

```
SELECT *
FROM employees
WHERE joining_date >= '2026-01-01'
  AND joining_date < '2027-01-01';
```

The second query uses a range condition on the original column.

Important: Some functions and expressions can be optimized through functional indexes or generated columns, but a simple range condition is often easier to optimize.

Q30. How can you identify redundant or unused indexes?

Answer: Review index definitions, query plans, and index usage statistics.

Useful tools include:

```
SHOW INDEX FROM employees;
```

And, where supported and permitted:

```
SELECT *
FROM sys.schema_unused_indexes;
```

You can also inspect Performance Schema index usage information.

Important: Usage statistics may reset after a restart or monitoring reset, and an index may be important for occasional reports, constraint enforcement, or operational tasks. Verify the workload over a representative period before dropping an index.

Q31. How would you optimize a slow query joining several large tables?

Answer: I would investigate the execution plan first, then optimize the most expensive operations.

A practical approach:

1. Run EXPLAIN and, when safe, EXPLAIN ANALYZE.
2. Identify large scans, expensive joins, and inaccurate row estimates.
3. Add or adjust indexes on relevant join and filter columns.
4. Apply selective filters as early as appropriate.
5. Avoid unnecessary columns and SELECT *.
6. Check whether join conditions use compatible data types.
7. Review sorting, grouping, temporary tables, and repeated subqueries.
8. Refresh statistics when appropriate and compare plans after changes.

Real-time scenario: An order-reporting query joining customers, orders, and order items may benefit from indexes on customer IDs, order IDs, and selective order-date conditions.

The best solution depends on the actual execution plan and data distribution.

Q32. How would you design indexes for a table containing millions of orders?

Answer: I would start by identifying the application's most frequent and important queries, then create indexes that support those access patterns.

For example, an orders table might contain:

- order_id — primary key.
- customer_id — customer order history.
- order_date — date-based searches and reporting.
- status — order-status filtering.
- A composite index such as (customer_id, order_date) — customer-specific order history.

Example:

```
CREATE INDEX idx_orders_customer_date

ON orders(customer_id, order_date);
```

For a query such as:

```
SELECT order_id, order_date, status

FROM orders

WHERE customer_id = 101
```

ORDER BY order_date DESC

LIMIT 20;

The composite index may help locate and order the customer's recent orders efficiently.

Important: Avoid adding every possible index. Consider write volume, index size, selectivity, pagination, and whether a covering index is justified. Validate the design with representative data and EXPLAIN ANALYZE.

Topic 4: InnoDB Internals and Transaction Isolation

Q33. What is the difference between InnoDB and MyISAM?

Answer: InnoDB and MyISAM are MySQL storage engines with different transaction, locking, and integrity capabilities.

Feature	**InnoDB**	**MyISAM**
Transactions	Supported	Not supported
ACID properties	Supports transactional ACID guarantees	Does not provide transactional ACID guarantees
Locking	Row-level locking with other locking mechanisms	Table-level locking
Foreign keys	Supported	Not enforced
Crash recovery	Supports crash recovery	Does not provide equivalent transactional recovery
Typical use	Transactional applications	Legacy or specialized read-oriented workloads

Real-time scenario: A banking application generally uses InnoDB because account transfers require transactions, rollback, and reliable concurrent updates.

InnoDB is the default storage engine in modern MySQL versions.

Q34. What are redo logs and undo logs in InnoDB?

Answer: Redo logs and undo logs serve different purposes in transaction processing and recovery.

- Redo logs: Record information needed to reapply changes during crash recovery.
- Undo logs: Preserve information needed to reverse uncommitted changes and provide older row versions for consistent reads.

Example: During a bank transfer, redo logging helps recover committed changes after a crash, while undo information allows an uncommitted transaction to be rolled back.

Interview point: Redo logging supports durability and recovery, while undo logging supports rollback and multi-version concurrency control (MVCC).

Q35. What is the InnoDB buffer pool, and how does it improve performance?

Answer: The InnoDB buffer pool is a memory area used to cache frequently accessed database pages, including table and index pages.

When requested pages are already in memory, InnoDB can often avoid reading them from disk. Modified pages are eventually written back to persistent storage.

Benefits:

- Reduces disk I/O.
- Speeds up repeated queries.
- Caches frequently accessed indexes and table data.
- Helps improve overall database throughput.

Real-time scenario: If an application repeatedly queries recent orders, keeping the relevant table and index pages in the buffer pool can reduce disk reads.

The appropriate buffer pool size depends on available memory and the needs of the operating system and other processes.

Q36. What is MVCC, and how does InnoDB use it?

Answer: MVCC stands for Multi-Version Concurrency Control. It allows transactions to access appropriate versions of rows without requiring every read to block concurrent writes.

InnoDB uses row-version information and undo records to provide consistent reads according to the transaction's isolation level and read view.

Benefits:

- Supports consistent reads.
- Reduces conflicts between many ordinary reads and writes.
- Helps implement transaction isolation.
- Allows older row versions to remain available while required by active transactions.

Real-time scenario: While one transaction updates an employee's salary, another transaction performing a consistent read can see the version permitted by its read view rather than necessarily waiting for the update to finish.

MVCC does not eliminate all locking. Writes and locking reads still require appropriate locks.

Q37. What are the transaction isolation levels supported by MySQL?

Answer: InnoDB supports four standard transaction isolation levels.

Isolation level	General behavior
READ UNCOMMITTED	Allows the weakest isolation and may permit dirty reads
READ COMMITTED	Each consistent read obtains a fresh read view
REPEATABLE READ	Consistent reads within a transaction generally use the same read view
SERIALIZABLE	Provides stronger isolation by imposing additional restrictions and locking behavior

InnoDB's default isolation level is generally REPEATABLE READ.

Check the configured isolation level:

SELECT @@transaction_isolation;

Set the isolation level for the next transaction:

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

The exact locking behavior also depends on whether an operation is a consistent read, a locking read, or a data modification.

Q38. What are dirty reads, non-repeatable reads, and phantom reads?

Answer: These are anomalies that can occur when concurrent transactions access the same data.

- Dirty read: A transaction reads another transaction's uncommitted changes.
- Non-repeatable read: A transaction reads a row twice and gets different committed values because another transaction updated it.
- Phantom read: Repeating a query for a range of rows produces a different set of matching rows because another transaction inserted or deleted qualifying rows.

Example: A transaction reads all orders above ₹10,000. Another transaction inserts a qualifying order and commits. Depending on the isolation level and type of read, the first transaction may see an additional matching row when it repeats the query.

InnoDB's default REPEATABLE READ provides consistent snapshots for ordinary consistent reads, while locking reads and writes can involve next-key locks and other concurrency mechanisms.

Q39. What is the difference between shared locks and exclusive locks?

Answer:

- Shared lock (S lock): Allows compatible transactions to hold shared locks on the same record, but prevents conflicting exclusive locks.
- Exclusive lock (X lock): Prevents other transactions from acquiring conflicting shared or exclusive locks on the same record.

Example: A transaction can acquire a shared lock using a locking read:

```
SELECT *
FROM accounts
WHERE account_id = 101
FOR SHARE;
```

An exclusive locking read can be performed using:

```
SELECT *
FROM accounts
WHERE account_id = 101
FOR UPDATE;
```

Shared locks are useful when a transaction needs to read records while preventing conflicting changes. Exclusive locks are used when records must be protected for modification.

Important: InnoDB can lock index records and ranges, not just abstract rows. The precise locks acquired depend on the query plan, indexes, and isolation level.

Q40. What is the difference between SELECT ... FOR UPDATE and SELECT ... FOR SHARE?

Answer: Both are locking reads in InnoDB, but they request different types of locks.

FOR UPDATE	**FOR SHARE**
Requests exclusive locks on the selected records	Requests shared locks on the selected records

Suitable when the transaction intends to modify the records	Suitable when the transaction needs to read records while preventing conflicting updates
Conflicts with other transactions' incompatible shared or exclusive locks	Allows compatible shared locks but conflicts with exclusive locks

Example using FOR UPDATE:

START TRANSACTION;

SELECT balance

FROM accounts

WHERE account_id = 101

FOR UPDATE;

-- Validate the balance and perform the required update.

COMMIT;

This is useful when a transaction needs to read an account balance and then update it safely.

Example using FOR SHARE:

START TRANSACTION;

SELECT account_id, balance

FROM accounts

WHERE account_id = 101

FOR SHARE;

COMMIT;

This reads the account while holding shared locks until the transaction ends.

Real-time scenario: A payment operation may use FOR UPDATE when it must safely validate and change an account balance. A workflow that needs a stable read while preventing concurrent modifications may use FOR SHARE.

Interview point: Both are transactional locking reads, and their behavior depends on the transaction isolation level and query plan. Use appropriate indexes, keep transactions short, and handle deadlocks when designing concurrent applications.

SQL Interview Questions and Answers (Q1–Q12)

1. Filtering and Sorting

Q1. Find employees earning more than the average salary.

Question: Display employees whose salary is greater than the company's average salary.

Answer:

```
SELECT emp_id, emp_name, salary
FROM employees
WHERE salary > (
   SELECT AVG(salary)
   FROM employees
);
```

Explanation: The subquery calculates the average salary, and the outer query retrieves employees earning more than that average.

Q2. Find the second-highest distinct salary without using LIMIT.

Answer:

```
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (
   SELECT MAX(salary)
   FROM employees
);
```

Explanation: First, the subquery finds the highest salary. The outer query then finds the maximum salary below it. Duplicate salary values are handled correctly.

Q3. Find employees who joined in the current year.

Answer:

```
SELECT emp_id, emp_name, hire_date
FROM employees
WHERE hire_date >= MAKEDATE(YEAR(CURDATE()), 1)
  AND hire_date < MAKEDATE(YEAR(CURDATE()) + 1, 1);
```

Explanation: This retrieves employees hired from January 1 of the current year up to, but not including, January 1 of the next year. It also allows efficient use of an index on hire_date.

Q4. Find employees whose names start with 'A' and end with 'n'.

Answer:

```
SELECT emp_id, emp_name
FROM employees
WHERE emp_name LIKE 'A%n';
```

Explanation: A% matches names starting with A, while the final n requires the name to end with n. % matches zero or more characters.

Q5. Display the top five highest-paid employees.

Answer:

```
SELECT emp_id, emp_name, salary
FROM employees
ORDER BY salary DESC, emp_id ASC
LIMIT 5;
```

Explanation: Employees are sorted by salary from highest to lowest. emp_id breaks salary ties consistently. The query returns at most five employees.

2. Aggregate Functions and Grouping

Q6. Find the highest salary in each department.

Answer:

```
SELECT department_id,
    MAX(salary) AS highest_salary
FROM employees
```

GROUP BY department_id;

Explanation: GROUP BY creates a group for each department, and MAX() returns the highest salary in each group.

Q7. Find departments with more than five employees.

Answer:

```
SELECT department_id,
    COUNT(*) AS employee_count
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 5;
```

Explanation: HAVING filters groups after aggregation, so only departments with more than five employees are displayed.

Q8. Find departments whose average salary exceeds ₹50,000.

Answer:

```
SELECT department_id,
    AVG(salary) AS average_salary
FROM employees
GROUP BY department_id
HAVING AVG(salary) > 50000;
```

Explanation: AVG() calculates each department's average salary. HAVING filters the departments whose average exceeds 50,000.

Q9. Find duplicate email addresses.

Answer:

```
SELECT email,
    COUNT(*) AS occurrences
FROM employees
WHERE email IS NOT NULL
GROUP BY email
HAVING COUNT(*) > 1;
```

Explanation: The query groups identical non-NULL email addresses and returns those appearing more than once.

Q10. Find employees earning more than their department's average salary.

Answer:

```
SELECT e.emp_id,
    e.emp_name,
    e.department_id,
    e.salary
FROM employees e
WHERE e.salary > (
  SELECT AVG(e2.salary)
  FROM employees e2
  WHERE e2.department_id = e.department_id
);
```

Explanation: The correlated subquery calculates the average salary for the current employee's department. The outer query selects employees earning more than that average.

3. Joins and Relationships

Q11. Display employee names along with their department names.

Answer:

```
SELECT e.emp_name,
    d.department_name
FROM employees e
INNER JOIN departments d
  ON e.department_id = d.department_id;
```

Explanation: INNER JOIN returns employees whose department IDs match records in the departments table.

Q12. Find employees who are not assigned to any department.

Answer:

```
SELECT e.emp_id, e.emp_name, e.department_id
FROM employees e
LEFT JOIN departments d
   ON e.department_id = d.department_id
WHERE e.department_id IS NULL
  OR d.department_id IS NULL;
```

Explanation: This includes employees whose department ID is NULL and employees whose non-NULL department ID has no matching department record.

Q13. Find departments that have no employees.

Answer:

```
SELECT d.department_id,
     d.department_name
FROM departments d
LEFT JOIN employees e
   ON d.department_id = e.department_id
WHERE e.emp_id IS NULL;
```

Explanation: The LEFT JOIN preserves every department. When no employee matches a department, the employee columns are NULL.

Q14. Display employees and their managers.

Answer:

```
SELECT e.emp_name AS employee_name,
     m.emp_name AS manager_name
FROM employees e
LEFT JOIN employees m
   ON e.manager_id = m.emp_id;
```

Explanation: This is a self-join because the same table represents both employees and managers. LEFT JOIN also includes employees without a manager.

Q15. Find customers who have never placed an order.

Answer:

```
SELECT c.customer_id,
       c.customer_name
FROM customers c
LEFT JOIN orders o
   ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Explanation: Customers without matching orders have NULL values in the order columns. The query returns only those customers.

4. Subqueries and Advanced Query Logic

Q16. Find employees earning more than their managers.

Answer:

```
SELECT e.emp_id,
       e.emp_name,
       e.salary,
       m.emp_name AS manager_name,
       m.salary AS manager_salary
FROM employees e
JOIN employees m
   ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

Explanation: The self-join matches each employee with their manager. The WHERE clause compares their salaries.

Q17. Find the third-highest distinct salary without using LIMIT.

Answer:

```
SELECT MAX(salary) AS third_highest_salary
FROM employees
WHERE salary < (
   SELECT MAX(salary)
   FROM employees
```

```
  WHERE salary < (
    SELECT MAX(salary)
    FROM employees
  )
);
```

Explanation: The innermost query finds the highest salary, the next query finds the second-highest, and the outer query finds the third-highest distinct salary. If fewer than three distinct non-NULL salaries exist, the result is NULL.

Q18. Find the highest-paid employee in each department, including ties.

Answer:

```
SELECT e.emp_id,
    e.emp_name,
    e.department_id,
    e.salary
FROM employees e
WHERE e.salary = (
  SELECT MAX(e2.salary)
  FROM employees e2
  WHERE e2.department_id = e.department_id
);
```

Explanation: Each employee's salary is compared with their department's maximum salary. All employees tied for the highest salary are returned.

Q19. Find products that have never been ordered.

Answer:

```
SELECT p.product_id,
    p.product_name
FROM products p
LEFT JOIN order_items oi
  ON p.product_id = oi.product_id
```

```
WHERE oi.product_id IS NULL;
```

Explanation: Products without a matching order item are returned.

Q20. Find customers who placed more than three orders in a month.

Answer:

```
SELECT customer_id,
    YEAR(order_date) AS order_year,
    MONTH(order_date) AS order_month,
    COUNT(*) AS total_orders
FROM orders
GROUP BY customer_id,
     YEAR(order_date),
     MONTH(order_date)
HAVING COUNT(*) > 3;
```

Explanation: Orders are grouped by customer, year, and month. Only customer-month groups containing more than three orders are returned.

5. Real-World Business Scenarios

Q21. Calculate monthly sales revenue.

Answer:

```
SELECT YEAR(order_date) AS sales_year,
    MONTH(order_date) AS sales_month,
    SUM(amount) AS total_revenue
FROM orders
GROUP BY YEAR(order_date), MONTH(order_date)
ORDER BY sales_year, sales_month;
```

Explanation: SUM() calculates revenue for each month, while YEAR() and MONTH() group the orders by calendar period.

Q22. Calculate month-over-month sales growth.

Answer:

```
WITH monthly_sales AS (
```

```
    SELECT DATE_FORMAT(order_date, '%Y-%m') AS sales_month,
        SUM(amount) AS revenue
    FROM orders
    GROUP BY DATE_FORMAT(order_date, '%Y-%m')
),
sales_comparison AS (
    SELECT sales_month,
        revenue,
        LAG(revenue) OVER (
            ORDER BY sales_month
        ) AS previous_revenue
    FROM monthly_sales
)
SELECT sales_month,
    revenue,
    previous_revenue,
    ROUND(
        100.0 * (revenue - previous_revenue)
        / NULLIF(previous_revenue, 0),
        2
    ) AS growth_percentage
FROM sales_comparison
ORDER BY sales_month;
```

Explanation: LAG() retrieves the revenue from the preceding row. NULLIF() prevents division by zero.

Note: This compares consecutive months present in the data. To calculate true calendar month-over-month growth when some months have no orders, generate a complete monthly calendar and treat missing sales as zero.

Q23. Find the best-selling product.

Answer:

```
SELECT p.product_id,
    p.product_name,
    SUM(oi.quantity) AS total_quantity_sold
FROM products p
JOIN order_items oi
  ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name
ORDER BY total_quantity_sold DESC
LIMIT 1;
```

Explanation: The query calculates the total quantity sold for each product and returns the top product. If multiple products tie for first place, this query returns only one.

Q24. Identify customers who made purchases in consecutive months.

Answer:

```
WITH customer_months AS (
  SELECT DISTINCT
      customer_id,
      DATE_FORMAT(order_date, '%Y-%m-01') AS order_month
  FROM orders
)
SELECT DISTINCT
    a.customer_id,
    a.order_month AS first_month,
    b.order_month AS next_month
FROM customer_months a
JOIN customer_months b
  ON a.customer_id = b.customer_id
 AND STR_TO_DATE(b.order_month, '%Y-%m-%d') =
```

```
    DATE_ADD(
      STR_TO_DATE(a.order_month, '%Y-%m-%d'),
      INTERVAL 1 MONTH
    );
```

Explanation: The query first creates one record per customer per month, then matches each month with the immediately following calendar month.

Q25. Calculate each department's percentage contribution to total salary expenditure.

Answer:

```
SELECT department_id,
    SUM(salary) AS department_salary,
    ROUND(
      100.0 * SUM(salary) /
      NULLIF((SELECT SUM(salary)
            FROM employees), 0),
      2
    ) AS contribution_percentage
FROM employees
GROUP BY department_id;
```

Explanation: The numerator is each department's total salary, and the denominator is the company's total salary expenditure. NULLIF() avoids division by zero.

6. Data Modification and Integrity

Q26. Increase salaries by 10% for employees in the IT department.

Answer:

```
UPDATE employees e
JOIN departments d
   ON e.department_id = d.department_id
SET e.salary = e.salary * 1.10
WHERE d.department_name = 'IT';
```

Explanation: The JOIN identifies employees belonging to IT, and the UPDATE increases only their salaries by 10%.

Q27. Delete duplicate records while keeping one record per email.

Answer:

```
DELETE e1
FROM employees e1
JOIN employees e2
   ON e1.email = e2.email
  AND e1.emp_id > e2.emp_id
WHERE e1.email IS NOT NULL;
```

Explanation: For each duplicated non-NULL email, the query deletes records with a higher emp_id, keeping the record with the smallest emp_id.

Before deleting, preview the affected rows using a corresponding SELECT. Back up the data and use a transaction where supported. If email uniqueness is a business requirement, add a suitable unique constraint after resolving duplicates.

Q28. Transfer money between two bank accounts.

Answer:

```
START TRANSACTION;


UPDATE accounts
SET balance = balance - 5000
WHERE account_id = 101
  AND balance >= 5000;


-- Verify that exactly one row was updated.
-- If not, execute ROLLBACK and stop.


UPDATE accounts
SET balance = balance + 5000
```

```
WHERE account_id = 202;


-- Verify that exactly one row was updated.
-- If not, execute ROLLBACK and stop.


COMMIT;
```

Explanation: The example transfers ₹5,000 from account 101 to account 202. Application logic must check the affected-row count after each update and execute ROLLBACK if either update fails. Both operations should use transactional tables such as InnoDB.

- START TRANSACTION begins the transaction.
- COMMIT permanently saves both successful changes.
- ROLLBACK reverses the transaction if an operation fails.

Q29. Update product stock only if sufficient stock is available.

Answer:

```
UPDATE products
SET stock_quantity = stock_quantity - 5
WHERE product_id = 101
  AND stock_quantity >= 5;
```

Explanation: Stock decreases by five only when the product exists and has sufficient stock. Check the affected-row count to determine whether the update succeeded.

Q30. Find employees whose salary is missing.

Answer:

```
SELECT emp_id, emp_name, salary
FROM employees
WHERE salary IS NULL;
```

Explanation: NULL represents a missing or unknown value. Use IS NULL, not = NULL, because comparisons with NULL produce an unknown result rather than TRUE.

7. Advanced JOIN Scenarios

Q31. Find employees who do not belong to any existing department.

Answer:

```
SELECT e.emp_id,
      e.emp_name,
      e.department_id
FROM employees e
LEFT JOIN departments d
   ON e.department_id = d.department_id
WHERE d.department_id IS NULL;
```

Explanation: The query returns employees with NULL department IDs and employees whose department IDs do not match an existing department.

Q32. Find customers who placed orders but never made a successful payment.

Answer:

```
SELECT c.customer_id,
      c.customer_name
FROM customers c
WHERE EXISTS (
   SELECT 1
   FROM orders o
   WHERE o.customer_id = c.customer_id
)
AND NOT EXISTS (
   SELECT 1
   FROM orders o
   JOIN payments p
      ON p.order_id = o.order_id
   WHERE o.customer_id = c.customer_id
     AND p.status = 'SUCCESS'
);
```

Explanation: EXISTS ensures the customer has placed an order. NOT EXISTS ensures none of their orders has a successful payment. A customer is included even if some payments failed or remain pending, provided no order has a successful payment.

Q33. Find products purchased by at least three different customers.

Answer:

```
SELECT p.product_id,
    p.product_name,
    COUNT(DISTINCT o.customer_id) AS customer_count
FROM products p
JOIN order_items oi
  ON p.product_id = oi.product_id
JOIN orders o
  ON oi.order_id = o.order_id
GROUP BY p.product_id, p.product_name
HAVING COUNT(DISTINCT o.customer_id) >= 3;
```

Explanation: COUNT(DISTINCT o.customer_id) counts unique customers rather than orders or quantities.

Q34. Find employees working on more than two projects.

Answer:

```
SELECT e.emp_id,
    e.emp_name,
    COUNT(DISTINCT ep.project_id) AS project_count
FROM employees e
JOIN employee_projects ep
  ON e.emp_id = ep.emp_id
GROUP BY e.emp_id, e.emp_name
HAVING COUNT(DISTINCT ep.project_id) > 2;
```

Explanation: The query counts distinct projects assigned to each employee and returns employees assigned to at least three projects.

Q35. Find pairs of employees in the same department who earn different salaries.

Answer:

```
SELECT e1.emp_name AS employee1,
       e2.emp_name AS employee2,
       e1.department_id
FROM employees e1
JOIN employees e2
    ON e1.department_id = e2.department_id
   AND e1.emp_id < e2.emp_id
WHERE e1.salary <> e2.salary;
```

Explanation: The self-join compares employees within the same department. The condition e1.emp_id < e2.emp_id prevents duplicate pairs and self-pairing. Employees with NULL salaries are excluded from the comparison.

Q36. Find customers who purchased products from every product category.

Answer:

```
SELECT c.customer_id,
       c.customer_name
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM categories cat
    WHERE NOT EXISTS (
        SELECT 1
        FROM orders o
        JOIN order_items oi
            ON o.order_id = oi.order_id
        JOIN products p
            ON oi.product_id = p.product_id
        WHERE o.customer_id = c.customer_id
```

```
        AND p.category_id = cat.category_id
    )
);
```

Explanation: The nested NOT EXISTS checks for categories for which a customer has no purchase. A customer qualifies when no such category exists. If the categories table is empty, this query returns every customer because there are no categories to miss.

8. Subqueries and Conditional Scenarios

Q37. Find employees whose salary is greater than every employee's salary in department 10.

Answer:

```
SELECT emp_id, emp_name, salary
FROM employees
WHERE salary > ALL (
   SELECT salary
   FROM employees
   WHERE department_id = 10
     AND salary IS NOT NULL
);
```

Explanation: > ALL requires the employee's salary to exceed every non-NULL salary returned by the subquery. If department 10 has no non-NULL salaries, the condition is true for all non-NULL employee salaries; handle that edge case separately if needed.

Q38. Find employees whose salary matches an employee's salary in another department.

Answer:

```
SELECT DISTINCT
      e1.emp_id,
      e1.emp_name,
      e1.department_id,
      e1.salary
FROM employees e1
```

```
JOIN employees e2
   ON e1.salary = e2.salary
  AND e1.department_id <> e2.department_id
WHERE e1.salary IS NOT NULL;
```

Explanation: The self-join finds employees who share a salary with someone in another department. DISTINCT prevents repeated output when multiple matching employees exist.

Q39. Find products priced above their category's average price.

Answer:

```
SELECT p.product_id,
      p.product_name,
      p.category_id,
      p.price
FROM products p
WHERE p.price > (
   SELECT AVG(p2.price)
   FROM products p2
   WHERE p2.category_id = p.category_id
);
```

Explanation: The correlated subquery calculates the average price within each product's category. Products priced above that average are returned.

Q40. Find customers whose total spending exceeds average customer spending.

Answer:

```
WITH customer_spending AS (
   SELECT customer_id,
        SUM(amount) AS total_spending
   FROM orders
   GROUP BY customer_id
)
SELECT customer_id,
```

```
    total_spending
FROM customer_spending
WHERE total_spending > (
  SELECT AVG(total_spending)
  FROM customer_spending
);
```

Explanation: The CTE calculates total spending per customer. The outer query compares each customer's spending against the average of those customer totals, not the average individual order amount.

Q41. Find departments where every employee earns more than ₹25,000.

Answer:

```
SELECT d.department_id,
    d.department_name
FROM departments d
WHERE EXISTS (
  SELECT 1
  FROM employees e
  WHERE e.department_id = d.department_id
)
AND NOT EXISTS (
  SELECT 1
  FROM employees e
  WHERE e.department_id = d.department_id
    AND (e.salary <= 25000 OR e.salary IS NULL)
);
```

Explanation: The first condition excludes departments with no employees. The second ensures there is no employee earning ₹25,000 or less and no employee with a missing salary.

Q42. Find the earliest order placed by each customer.

Answer:

```
SELECT o.customer_id,
    o.order_id,
    o.order_date
FROM orders o
WHERE o.order_date = (
  SELECT MIN(o2.order_date)
  FROM orders o2
  WHERE o2.customer_id = o.customer_id
);
```

Explanation: The subquery finds each customer's earliest order date. If multiple orders share that date, all are returned. Customers without orders do not appear because the query starts from orders.

9. Window Function Scenarios

Q43. Rank employees by salary within each department.

Answer:

```
SELECT emp_id,
    emp_name,
    department_id,
    salary,
    RANK() OVER (
      PARTITION BY department_id
      ORDER BY salary DESC
    ) AS salary_rank
FROM employees;
```

Explanation: PARTITION BY starts a separate ranking for each department. RANK() assigns equal ranks to employees with the same salary and skips subsequent rank numbers after ties.

Q44. Find the top three salary-ranking positions in each department, including ties.

Answer:

```
WITH ranked_employees AS (
   SELECT emp_id,
        emp_name,
        department_id,
        salary,
        DENSE_RANK() OVER (
           PARTITION BY department_id
           ORDER BY salary DESC
        ) AS salary_rank
   FROM employees
)
SELECT emp_id,
      emp_name,
      department_id,
      salary,
      salary_rank
FROM ranked_employees
WHERE salary_rank <= 3;
```

Explanation: DENSE_RANK() assigns a distinct rank to each distinct salary without gaps. All employees tied within the top three salary ranks are included, so the query may return more than three employees per department.

Q45. Calculate a running total of sales.

Answer:

```
SELECT order_id,
      order_date,
      amount,
      SUM(amount) OVER (
         ORDER BY order_date, order_id
```

```
        ROWS BETWEEN UNBOUNDED PRECEDING
            AND CURRENT ROW
    ) AS running_total
FROM orders;
```

Explanation: The window function accumulates the order amounts chronologically. order_id provides deterministic ordering when multiple orders share the same date.

Q46. Find the salary difference from the previous employee in the same department.

Answer:

```
SELECT emp_id,
    emp_name,
    department_id,
    salary,
    salary - LAG(salary) OVER (
        PARTITION BY department_id
        ORDER BY salary, emp_id
    ) AS salary_difference
FROM employees;
```

Explanation: LAG() retrieves the previous employee's salary in ascending salary order within each department. The first employee in each department has a NULL difference.

Q47. Identify the first and most recent order for every customer.

Answer:

```
SELECT c.customer_id,
    c.customer_name,
    MIN(o.order_date) AS first_order_date,
    MAX(o.order_date) AS most_recent_order_date
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name;
```

Explanation: MIN() and MAX() find the earliest and latest order dates. LEFT JOIN retains customers who have never placed an order; their dates will be NULL.

Q48. Find consecutive duplicate salary values in salary history.

Answer:

```
WITH salary_history AS (
   SELECT emp_id,
        effective_date,
        salary,
        LAG(salary) OVER (
           PARTITION BY emp_id
           ORDER BY effective_date
        ) AS previous_salary
   FROM employee_salary_history
)
SELECT emp_id,
     effective_date,
     salary,
     previous_salary
FROM salary_history
WHERE salary = previous_salary;
```

Explanation: LAG() compares each salary with the immediately preceding salary record for the same employee. The query returns consecutive records with equal, non-NULL salary values.

10. Date and Time Scenarios

Q49. Find employees who have completed five years of service.

Answer:

```
SELECT emp_id,
     emp_name,
     hire_date
```

```
FROM employees
WHERE hire_date <= DATE_SUB(CURDATE(), INTERVAL 5 YEAR);
```

Explanation: The query returns employees whose hire date is on or before the date five years ago. Their five-year anniversary has occurred.

Q50. Calculate the number of days between consecutive orders for each customer.

Answer:

```
SELECT customer_id,
    order_id,
    order_date,
    DATEDIFF(
      order_date,
      LAG(order_date) OVER (
        PARTITION BY customer_id
        ORDER BY order_date, order_id
      )
    ) AS days_since_previous_order
FROM orders;
```

Explanation: LAG() retrieves the previous order date for each customer, and DATEDIFF() calculates the difference in days. The first order for each customer has a NULL result.

Q51. Find orders placed on weekends.

Answer:

```
SELECT order_id,
    order_date,
    amount
FROM orders
WHERE DAYOFWEEK(order_date) IN (1, 7);
```

Explanation: In MySQL, DAYOFWEEK() returns 1 for Sunday and 7 for Saturday, so the condition selects weekend orders.

Q52. Find customers who have not ordered in the last 90 days, including those who never ordered.

Answer:

```
SELECT c.customer_id,
      c.customer_name
FROM customers c
WHERE NOT EXISTS (
   SELECT 1
   FROM orders o
   WHERE o.customer_id = c.customer_id
     AND o.order_date >= DATE_SUB(CURDATE(), INTERVAL 90 DAY)
     AND o.order_date < CURDATE() + INTERVAL 1 DAY
);
```

Explanation: The query excludes customers who have any order from the beginning of the 90-day window through today. Customers who have never ordered are included.

Q53. Calculate quarterly sales revenue.

Answer:

```
SELECT YEAR(order_date) AS sales_year,
      QUARTER(order_date) AS sales_quarter,
      SUM(amount) AS total_revenue
FROM orders
GROUP BY YEAR(order_date),
       QUARTER(order_date)
ORDER BY sales_year, sales_quarter;
```

Explanation: QUARTER() returns 1 through 4 for the calendar quarter. Grouping by year as well prevents different years' quarters from being combined.

Q54. Find the busiest sales day in each month, including ties.

Answer:

```
WITH daily_orders AS (
```

```
    SELECT DATE(order_date) AS sales_date,
        COUNT(*) AS order_count
    FROM orders
    GROUP BY DATE(order_date)
),
ranked_days AS (
    SELECT sales_date,
        order_count,
        RANK() OVER (
            PARTITION BY YEAR(sales_date),
                MONTH(sales_date)
            ORDER BY order_count DESC
        ) AS day_rank
    FROM daily_orders
)
SELECT sales_date,
    order_count
FROM ranked_days
WHERE day_rank = 1
ORDER BY sales_date;
```

Explanation: First, the query counts orders per calendar day. Then RANK() identifies the busiest day within each year-month combination. If several days share the highest order count, all are returned.